**Bahria University Islamabad**

Department of Computer Science

Faculty of Computing and Information Technology

# NovaCORE

*An OS Concepts Simulator with xv6 Kernel Extension*

**Course:** Operating Systems (CSC320)

**Instructor:** Ms. Sidra Ayesha

**Section:** BSIT 5A **Semester:** Spring 2026

## Group Members



| blue!10 # | **Name** | **Enrollment No.** | **Module** |
|---|---|---|---|
| 1 | Muhammad Ibrahim | 01-135241-035 | Process Scheduler & Main Menu |
| 2 | Manzer Bibi | 01-135241-026 | Memory Manager |
| 3 | Abdul Wahab | 01-135241-001 | Deadlock & Process State |
| 4 | Ateeq Ur Rehman | 01-135241-011 | IPC Producer-Consumer |
| 5 | Saim Zafar | 01-135241-045 | xv6 Kernel Extension |



**Submission Date:** 13 May 2026

**Submission Mode:** LMS Upload (PDF Report + Source Code)

# Contents


**1 Introduction** **5**
- 1.1 Background . . . 5
- 1.2 What is NovaCORE? . . . 5
- 1.3 Modules Overview . . . 6
- 1.4 Technologies Used . . . 6
- 1.5 Project Goals . . . 6

**2 Problem Statement** **8**
- 2.1 The Core Problem . . . 8
- 2.2 Specific Problems Identified . . . 8
- 2.3 Why Existing Tools Were Not Enough . . . 9
- 2.4 The Solution . . . 9

**3 Methodology and Algorithms Used** **10**
- 3.1 Process Scheduling Algorithms . . . 10
  - 3.1.1 Round Robin (RR) . . . 10
  - 3.1.2 Shortest Job First — Non-Preemptive (SJF) . . . 11
  - 3.1.3 Priority Scheduling — Non-Preemptive . . . 11
- 3.2 Page Replacement Algorithms . . . 11
  - 3.2.1 FIFO — First In First Out . . . 12
  - 3.2.2 LRU — Least Recently Used . . . 12
  - 3.2.3 Optimal Page Replacement . . . 12
- 3.3 Banker's Algorithm — Deadlock Detection . . . 12
- 3.4 Producer-Consumer with Semaphores . . . 13
- 3.5 Process State Simulation . . . 13
- 3.6 xv6 Kernel Extension — getmeminfo() Syscall . . . 14

**4 Flowchart and System Design** **15**

**5 Step-by-Step Module Working** **19**
- 5.1 Module 1 - Process Scheduler . . . 19
  - 5.1.1 Step-by-Step Working . . . 19
  - 5.1.2 Example Run . . . 20
- 5.2 Module 2 - Memory Manager . . . 20
  - 5.2.1 Step-by-Step Working . . . 20
  - 5.2.2 Example Run . . . 21
- 5.3 Module 3 - IPC Producer-Consumer . . . 21

5.3.1 Step-by-Step Working . . . 22
5.4 Module 4 - Process State Simulation . . . 23
5.4.1 Step-by-Step Working . . . 23
5.4.2 Example Run . . . 23
5.5 Module 5 - Deadlock Detection (Banker's Algorithm) . . . 23
5.5.1 Step-by-Step Working . . . 23
5.5.2 Preset Example Result . . . 24
5.6 Module 6 - xv6 Kernel Extension (getmeminfo) . . . 24
5.6.1 Step-by-Step Working . . . 25
5.6.2 Summary Table . . . 25

**6 Implementation Details 26**
6.1 Project Structure . . . 26
6.2 main.c — Entry Point and Menu . . . 27
6.3 scheduler.c — Process Scheduler . . . 27
6.3.1 Input Handling . . . 27
6.3.2 Round Robin Implementation . . . 27
6.3.3 SJF Implementation . . . 27
6.3.4 Priority Implementation . . . 28
6.3.5 Output . . . 28
6.4 memory.c — Page Replacement . . . 28
6.4.1 FIFO . . . 28
6.4.2 LRU . . . 28
6.4.3 Optimal . . . 28
6.4.4 Compare Mode . . . 28
6.5 ipc.c — Producer-Consumer . . . 29
6.5.1 Synchronization Design . . . 29
6.5.2 Thread Functions . . . 29
6.6 process_state.c — Process State Simulation . . . 29
6.7 deadlock.c — Banker's Algorithm . . . 30
6.7.1 Input Validation . . . 30
6.7.2 Need Matrix Computation . . . 30
6.7.3 Safety Algorithm . . . 30
6.7.4 Resource Allocation Graph . . . 30
6.8 xv6 Kernel Extension — Implementation . . . 30
6.8.1 struct meminfo in proc.h . . . 31
6.8.2 kalloc_free_pages() in kalloc.c . . . 31
6.8.3 sys_getmeminfo() in sysproc.c . . . 31
6.8.4 Syscall Registration . . . 31
6.8.5 GCC 13 Compatibility Fixes . . . 32
6.9 Makefile and Build System . . . 32

**7 Complete Code 33**
7.1 main.c — Entry Point and Menu Loop . . . 33
7.2 scheduler.c — Process Scheduling Algorithms . . . 34
7.2.1 Round Robin . . . 34
7.2.2 SJF — Non-Preemptive . . . 35
7.2.3 Priority — Non-Preemptive . . . 36

7.3 memory.c — Page Replacement Algorithms . . . 36
7.3.1 FIFO . . . 36
7.3.2 LRU . . . 37
7.3.3 Optimal . . . 38
7.4 ipc.c — Producer-Consumer with Semaphores . . . 38
7.5 process_state.c — Process State Simulation . . . 39
7.6 deadlock.c — Banker's Algorithm . . . 41
7.6.1 Need Matrix Computation . . . 41
7.6.2 Safety Algorithm . . . 41
7.6.3 Resource Allocation Graph (Text-Based) . . . 42
7.7 meminfo.c — xv6 User Utility . . . 42
7.8 xv6 Kernel Patches . . . 43
7.8.1 patch_kalloc_c.c — Free Page Counter . . . 43
7.8.2 patch_sysproc_c.c — Syscall Handler . . . 43
7.8.3 Syscall Registration . . . 44

**8 Results and Output Screenshots 45**
8.1 Module 1 — Process Scheduler . . . 45
8.1.1 Round Robin . . . 45
8.1.2 SJF — Non-Preemptive . . . 47
8.1.3 Priority — Non-Preemptive . . . 48
8.2 Module 2 — Memory Manager . . . 48
8.2.1 FIFO Page Replacement . . . 49
8.2.2 LRU Page Replacement . . . 50
8.2.3 Optimal Page Replacement . . . 51
8.2.4 Comparison of All Three Algorithms . . . 52
8.3 Module 3 — IPC Producer-Consumer . . . 52
8.4 Module 4 — Process State Simulation . . . 53
8.5 Module 5 — Deadlock Detection . . . 54
8.5.1 Resource State Table . . . 55
8.5.2 Safety Check and Safe Sequence . . . 55
8.5.3 Resource Allocation Graph . . . 56
8.6 Module 6 — xv6 Kernel Extension . . . 56
8.6.1 xv6 Boot . . . 56
8.6.2 meminfo Output . . . 56
8.7 NovaCORE Main Menu . . . 58

**9 Challenges and Solutions 59**
9.1 Challenge 1 — Round Robin Clock Initialization Bug . . . 59
9.1.1 The Problem . . . 59
9.1.2 The Solution . . . 59
9.2 Challenge 2 — xv6 Not Compiling on GCC 13 . . . 60
9.2.1 The Problem . . . 60
9.2.2 The Solution . . . 60
9.3 Challenge 3 — Kernel Pointer Validation with argptr() . . . 61
9.3.1 The Problem . . . 61
9.3.2 The Solution . . . 61
9.4 Challenge 4 — IPC Race Condition in Early Version . . . 61

9.4.1 The Problem . . . 61
9.4.2 The Solution . . . 62
9.5 Challenge 5 — Optimal Page Replacement Look-Ahead Logic . . . 62
9.5.1 The Problem . . . 62
9.5.2 The Solution . . . 62
9.6 Challenge 6 — Process State Simulation Infinite Loop . . . 63
9.6.1 The Problem . . . 63
9.6.2 The Solution . . . 63

**10 Individual Contribution** **64**
10.1 Muhammad Ibrahim — 01-135241-035 . . . 64
10.2 Manzer Bibi — 01-135241-026 . . . 64
10.3 Abdul Wahab — 01-135241-001 . . . 65
10.4 Ateeq Ur Rehman — 01-135241-011 . . . 65
10.5 Saim Zafar — 01-135241-045 . . . 66

**11 Conclusion** **67**
11.1 What Was Accomplished . . . 67
11.2 What Was Learned . . . 67
11.3 Course Learning Outcomes . . . 68
11.4 Limitations and Future Work . . . 68
11.5 Final Remarks . . . 69

# Chapter 1

# Introduction

## 1.1 Background

Operating Systems is one of those subjects where just reading theory is not enough. You can read about scheduling algorithms, memory management, and deadlock detection all you want, but unless you actually see them running and producing results, it is hard to really understand what is going on. That is the main reason this project was started.

Most students in OS courses face the same problem. The concepts taught in class, like Round Robin scheduling, page replacement, Banker's algorithm, and inter-process communication, are often only shown through textbook diagrams or small paper-based examples. There is rarely a chance to run these algorithms yourself, give your own input, and see what the output looks like in real time.

NovaCORE was built to solve exactly this problem. It is a terminal-based simulator written in C that brings all the major OS concepts together in one place. You can run different scheduling algorithms with your own process data, test page replacement strategies, simulate deadlock detection, watch process state transitions happen cycle by cycle, and test producer-consumer synchronization. Everything runs interactively through a simple menu.

On top of that, the project also includes a real kernel-level extension for xv6, which is a small Unix-like operating system used for teaching at MIT. A custom system call called `getmeminfo()` was added to the xv6 kernel, along with a user-space utility called `meminfo` that calls this syscall and prints live memory usage information from inside the running kernel. This makes NovaCORE a hybrid project, part simulator running on Linux, and part actual kernel modification running inside QEMU.

## 1.2 What is NovaCORE?

NovaCORE stands for the core concepts of an operating system brought together in one simulator. The name reflects the idea of covering the fundamental, central parts of OS theory in a hands-on way.

The project has two main parts:

1. **The Linux Simulator** — A C program that runs on any Linux machine. It has six modules, each covering a different OS concept. The user interacts with it through a menu, enters their own data, and sees the results immediately.
2. **The xv6 Kernel Extension** — A set of patches applied to the xv6 teaching kernel that add a new system call (`getmeminfo`, syscall number 22) and a user utility (`meminfo`). This runs inside QEMU and demonstrates real kernel-level programming.

## 1.3 Modules Overview

NovaCORE covers six core OS topics:

- **Process Scheduler** — Implements Round Robin, Shortest Job First (Non-Preemptive), and Priority Scheduling (Non-Preemptive). Calculates Completion Time, Turnaround Time, and Waiting Time for each process.
- **Memory Manager** — Implements FIFO, LRU, and Optimal page replacement algorithms. Shows frame-by-frame page hits and misses, and calculates the overall hit rate.
- **IPC Producer-Consumer** — Uses POSIX threads, semaphores, and mutexes to simulate a bounded buffer where producer and consumer threads work concurrently without race conditions.
- **Process State Simulation** — Simulates the five process states (NEW, READY, RUNNING, WAITING, TERMINATED) cycle by cycle, with I/O wait triggers built in.
- **Deadlock Detection** — Implements the Banker's Algorithm to check if the system is in a safe state, finds the safe sequence, and also generates a text-based Resource Allocation Graph.
- **xv6 Kernel Extension** — Adds a real `getmeminfo()` system call to the xv6 kernel that reads live physical memory statistics by walking the kernel's free page list.

## 1.4 Technologies Used

The entire simulator is written in **C (C99 standard)** and compiled using GCC. POSIX threads (`pthreads`) are used for the IPC module. The xv6 part runs inside **QEMU** (an x86 emulator) and was built using GCC with 32-bit compilation flags. The project compiles cleanly with `-Wall` and zero warnings on Ubuntu 24.04 with GCC 13.

## 1.5 Project Goals

The main goals of this project were:

- To implement core OS algorithms from scratch in C and verify their correctness.
- To make the simulator interactive so that anyone can test different inputs and observe results.

- To go beyond just simulation by adding a real system call to an actual teaching kernel (xv6).
- To cover the CLOs of the OS course by applying, analyzing, and implementing OS concepts at both user level and kernel level.

This project satisfies CLO1 by applying fundamental OS concepts and system-level programming, CLO2 by implementing process management, memory management, synchronization, and scheduling, and CLO3 by designing and developing OS-based solutions using appropriate algorithms and techniques.

# Chapter 2

# Problem Statement

## 2.1 The Core Problem

Operating Systems is a theory-heavy subject. Topics like CPU scheduling, page replacement, deadlock handling, and inter-process communication are all taught through lectures, textbook diagrams, and paper-based examples. While this helps build understanding, it has one big limitation: students never actually run these algorithms themselves.

When you solve a Round Robin scheduling problem on paper, you follow the steps manually and get an answer. But you do not get to change the time quantum and immediately see how results differ. You do not get to test five different sets of processes and compare outputs. The learning stops at a single static example.

The same applies to every other OS concept. Page replacement algorithms are shown with fixed reference strings. Deadlock detection is demonstrated with a preset matrix. Process states are explained with a diagram. None of these are interactive, and none of them let a student experiment freely.

## 2.2 Specific Problems Identified

After going through the OS course, we identified the following specific gaps:

- **No hands-on scheduling tool:** There was no simple program where you could enter your own process arrival times, burst times, and priorities, pick an algorithm, and immediately see the Completion Time, Turnaround Time, and Waiting Time calculated correctly.
- **No interactive page replacement simulator:** Page replacement algorithms like FIFO, LRU, and Optimal are hard to compare without running them side by side on the same input. Textbook examples only show one algorithm at a time.
- **No working deadlock detector:** The Banker's Algorithm is one of the more complex topics in OS. Without being able to run it on custom inputs, it is easy to make mistakes in the need matrix, the safety check loop, or the safe sequence calculation.

- **No visible process state transitions:** The five-state process model (NEW, READY, RUNNING, WAITING, TERMINATED) is explained in theory, but seeing it happen cycle by cycle for multiple processes at the same time makes it much clearer.
- **No practical IPC demonstration:** Producer-consumer synchronization using semaphores and mutexes is easy to get wrong. Race conditions and deadlocks can occur if the semaphore logic is even slightly off. There was no simple working example to study from.
- **No kernel-level OS work:** Everything in the OS lab was user-space programming. There was no experience with how the kernel actually works, how system calls are added, or how the kernel manages physical memory at the hardware level.

## 2.3 Why Existing Tools Were Not Enough

There are some online simulators for OS algorithms, but they have problems too. Most of them:

- Run in a browser and do not show actual C code behind the algorithm.
- Only support one or two algorithms, not all the major ones in one place.
- Do not let you compare algorithms on the same input easily.
- Have no connection to real kernel concepts at all.

We needed something that was written from scratch in C, ran in a terminal, showed real algorithm logic, and also touched actual kernel-level programming. That is what NovaCORE was designed to be.

## 2.4 The Solution

NovaCORE solves these problems in two parts:

**Part 1 — The Linux Simulator:** A terminal-based C program with six modules covering all the major OS concepts. Each module takes user input, runs the algorithm, and shows detailed output. The user can test any input they want and immediately see the results. All algorithms are implemented from scratch so the code itself is also a learning resource.

**Part 2 — The xv6 Kernel Extension:** To go beyond simulation, a real system call was added to the xv6 teaching kernel. The `getmeminfo()` syscall reads live physical memory data from inside the kernel by walking the free page linked list. A user utility called `meminfo` was also written and added to xv6 so it can be run from the xv6 shell directly. This gives hands-on experience with how system calls are registered, how kernel-user memory transfer works, and how physical memory is managed at the kernel level.

Together, these two parts cover OS concepts from both the application level and the kernel level, which is exactly the kind of complete understanding the course aims to build.

# Chapter 3

# Methodology and Algorithms Used

This chapter explains each algorithm that was implemented in NovaCORE, how it works, and why it was chosen. The focus is on the logic behind each one, not just a textbook definition.

## 3.1 Process Scheduling Algorithms

The scheduler module implements three non-preemptive CPU scheduling algorithms. All three sort processes by arrival time first, then apply their own selection logic.

The two key formulas used across all three algorithms are:

$$\text{Turnaround Time (TAT)} = \text{Completion Time} - \text{Arrival Time} \tag{3.1}$$

$$\text{Waiting Time (WT)} = \text{Turnaround Time} - \text{Burst Time} \tag{3.2}$$

### 3.1.1 Round Robin (RR)

Round Robin is one of the most commonly used scheduling algorithms for time-sharing systems. The idea is simple: every process gets a fixed slice of CPU time called the **time quantum**. If a process does not finish in its quantum, it goes back to the end of the ready queue and waits for its next turn.

**How it works in NovaCORE:**

1. Processes are sorted by arrival time.
2. The CPU time starts at the arrival time of the first process (not at zero), which avoids incorrect results when processes arrive late.
3. A circular queue holds the ready processes. When a process is picked, it runs for either the quantum or its remaining burst time, whichever is smaller.
4. After running, if the process still has burst time left, it goes back into the queue. New arrivals during that time are also added before it.
5. This continues until all processes are done.

**Why Round Robin:** It gives fair CPU time to all processes and is the standard algorithm for time-sharing. It also has the most interesting edge cases to implement correctly, especially around queue management and arrival time handling.

**Note:** An important bug was found and fixed during testing. The original code started the clock at time zero instead of the first process arrival time. This caused negative waiting times when all processes had arrival times greater than zero. The fix was to initialize `time = procs[0].arrival_time` and enqueue all processes that had already arrived at that point before starting the main loop.

### 3.1.2 Shortest Job First — Non-Preemptive (SJF)

In SJF, the process with the shortest burst time among all currently available processes is picked next. Once a process starts running, it runs to completion without interruption (non-preemptive).

**How it works in NovaCORE:**

1. At each step, all processes that have arrived by the current time are considered.
2. The one with the smallest burst time is selected.
3. If no process has arrived yet, the clock jumps forward to the next arrival time.
4. This repeats until all processes finish.

**Why SJF:** It gives the minimum average waiting time among all non-preemptive algorithms, which makes it a useful comparison point. It also demonstrates how knowing burst times in advance can improve scheduling efficiency.

### 3.1.3 Priority Scheduling — Non-Preemptive

In priority scheduling, each process has a priority number. The process with the lowest priority number (highest priority) is picked first from all available processes. Like SJF, once a process starts it runs to completion.

**How it works in NovaCORE:**

1. At each step, all arrived processes are checked.
2. The one with the smallest priority number is selected.
3. Ties are broken by picking the process that arrived first.
4. The clock advances to the next arrival if nothing is ready.

**Why Priority:** It shows how real-time and critical processes can be given preference over less important ones. It also pairs well with Round Robin, since some systems use priority to decide which process enters the RR queue.

## 3.2 Page Replacement Algorithms

The memory manager module simulates physical memory frames and shows what happens when a new page is requested but all frames are full (a page fault). Three algorithms are

implemented.

The hit rate is calculated as:

$$\text{Hit Rate} = \frac{\text{Total Hits}}{\text{Total Page References}} \times 100\% \tag{3.3}$$

### 3.2.1 FIFO — First In First Out

FIFO replaces the page that has been in memory the longest. It uses a circular pointer to track which frame was loaded first and replaces that one when a new page needs to come in.

**Why FIFO:** It is the simplest page replacement algorithm and is easy to implement. It is used as a baseline for comparing the other two algorithms.

### 3.2.2 LRU — Least Recently Used

LRU replaces the page that was used least recently. Each frame has a `last_used` timestamp. When a page fault occurs, the frame with the oldest (smallest) timestamp is replaced.

**Why LRU:** It performs better than FIFO in most real-world cases because it keeps recently used pages in memory, which matches the principle of temporal locality. It is also one of the most studied page replacement algorithms.

### 3.2.3 Optimal Page Replacement

The Optimal algorithm looks ahead in the reference string and replaces the page that will not be used for the longest time in the future. If a page in memory is never used again, it is the first candidate for replacement.

**Why Optimal:** It gives the best possible hit rate for any given reference string. It is not practical to implement in real operating systems because future page references are not known in advance, but it is useful as a theoretical upper bound for comparing other algorithms.

## 3.3 Banker's Algorithm — Deadlock Detection

The Banker's Algorithm is used to check if a system is in a safe state. A safe state means there exists at least one sequence in which all processes can finish without causing deadlock.

**Key data structures used:**

- **Allocation matrix** — how many resources each process currently holds.
- **Maximum matrix** — the maximum resources each process may ever need.
- **Need matrix** — computed as: Need = Maximum − Allocation
- **Available vector** — resources currently available in the system.

**Safety Algorithm steps:**

1. Set `Work = Available` and mark all processes as unfinished.
2. Find any unfinished process whose `Need` is less than or equal to `Work`.
3. If found, simulate its completion: add its `Allocation` back to `Work` and mark it finished.
4. Repeat steps 2–3 until no more processes can proceed.
5. If all processes are finished, the system is in a safe state. The order in which they were completed is the safe sequence.
6. If not all processes finished, the system is in an unsafe state and deadlock may occur.

NovaCORE also generates a text-based Resource Allocation Graph showing which processes are requesting which resources and which resources are allocated to which processes.

The preset example in the simulator uses the classic textbook scenario with 5 processes and 3 resource types, which produces the safe sequence: **P1** → **P3** → **P4** → **P0** → **P2**.

## 3.4 Producer-Consumer with Semaphores

The IPC module implements the classic bounded buffer problem using POSIX threads, semaphores, and mutexes.

**Synchronization tools used:**

- `empty_sem` — initialized to buffer size. Counts how many empty slots are available. A producer waits on this before producing.
- `full_sem` — initialized to 0. Counts how many filled slots exist. A consumer waits on this before consuming.
- Two mutexes — one for producers, one for consumers, to prevent simultaneous access to the buffer.

**How it works:**

1. Producer threads wait for an empty slot, lock the mutex, write to the buffer, unlock, and signal `full_sem`.
2. Consumer threads wait for a full slot, lock the mutex, read from the buffer, unlock, and signal `empty_sem`.
3. This ensures no producer overwrites an unconsumed item and no consumer reads an empty slot.

## 3.5 Process State Simulation

This module simulates the five standard process states from OS theory:

**NEW** → **READY** → **RUNNING** → **WAITING** → **TERMINATED**

**Rules used in the simulation:**

- Only one process runs at a time (simple dispatcher).
- Every time a running process hits a burst remaining value that is a multiple of 3, it triggers an I/O wait and moves to WAITING state for 2 cycles.
- After 2 cycles, it returns to READY.
- When burst time reaches zero, the process moves to TERMINATED.
- A safety cap of 60 cycles prevents infinite loops.

## 3.6 xv6 Kernel Extension — getmeminfo() Syscall

The xv6 extension adds a real system call to the teaching kernel. This was done by modifying multiple kernel source files.

**How the syscall works:**

1. The user program `meminfo` calls `getmeminfo(&mi)` where `mi` is a `struct meminfo` pointer.
2. The kernel function `sys_getmeminfo()` in `sysproc.c` receives this call.
3. It uses `argptr()` to safely copy the user-space pointer to kernel space.
4. It then calls `kalloc_free_pages()` in `kalloc.c`, which acquires the memory lock and walks the kernel's free page linked list to count free pages.
5. Total pages are calculated from the known physical memory layout of xv6.
6. The filled struct is returned to user space and `meminfo` prints the results.

**Files modified in xv6:**

- `syscall.h` — added syscall number 22 for `getmeminfo`
- `syscall.c` — registered the syscall handler
- `sysproc.c` — implemented `sys_getmeminfo()`
- `kalloc.c` — added `kalloc_free_pages()` function
- `defs.h` — added function prototype
- `user.h` — added user-space declaration
- `usys.S` — added syscall stub
- `proc.h` — added `struct meminfo` definition
- `Makefile` — added `meminfo` to the user program build list

# Chapter 4

# Flowchart and System Design

This chapter presents the system architecture and flowcharts for the major components of NovaCORE. All diagrams were drawn using TikZ.

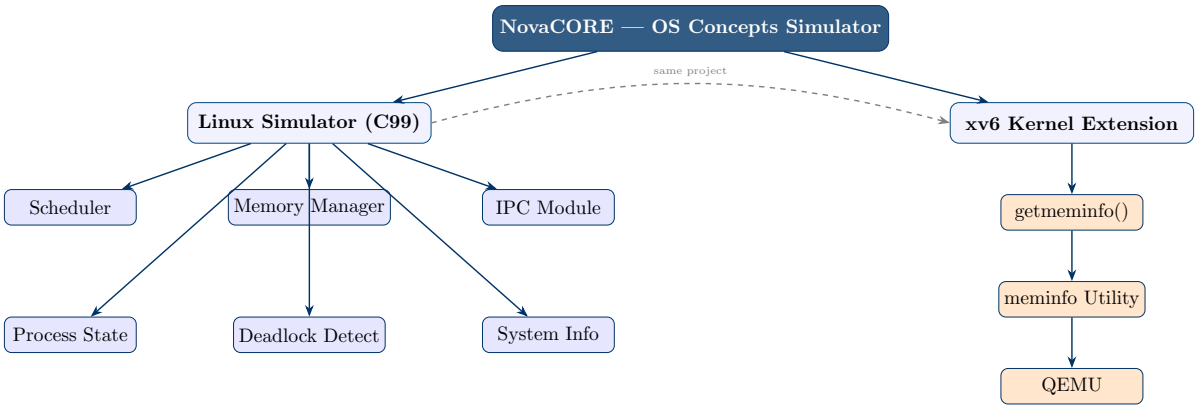


Figure 4.1: NovaCORE Overall System Architecture

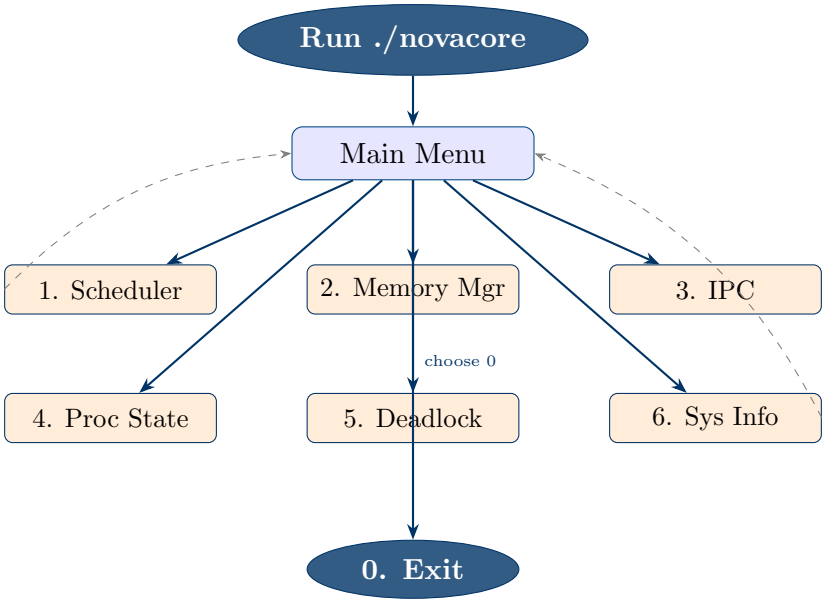


Figure 4.2: Main Menu Navigation Flow

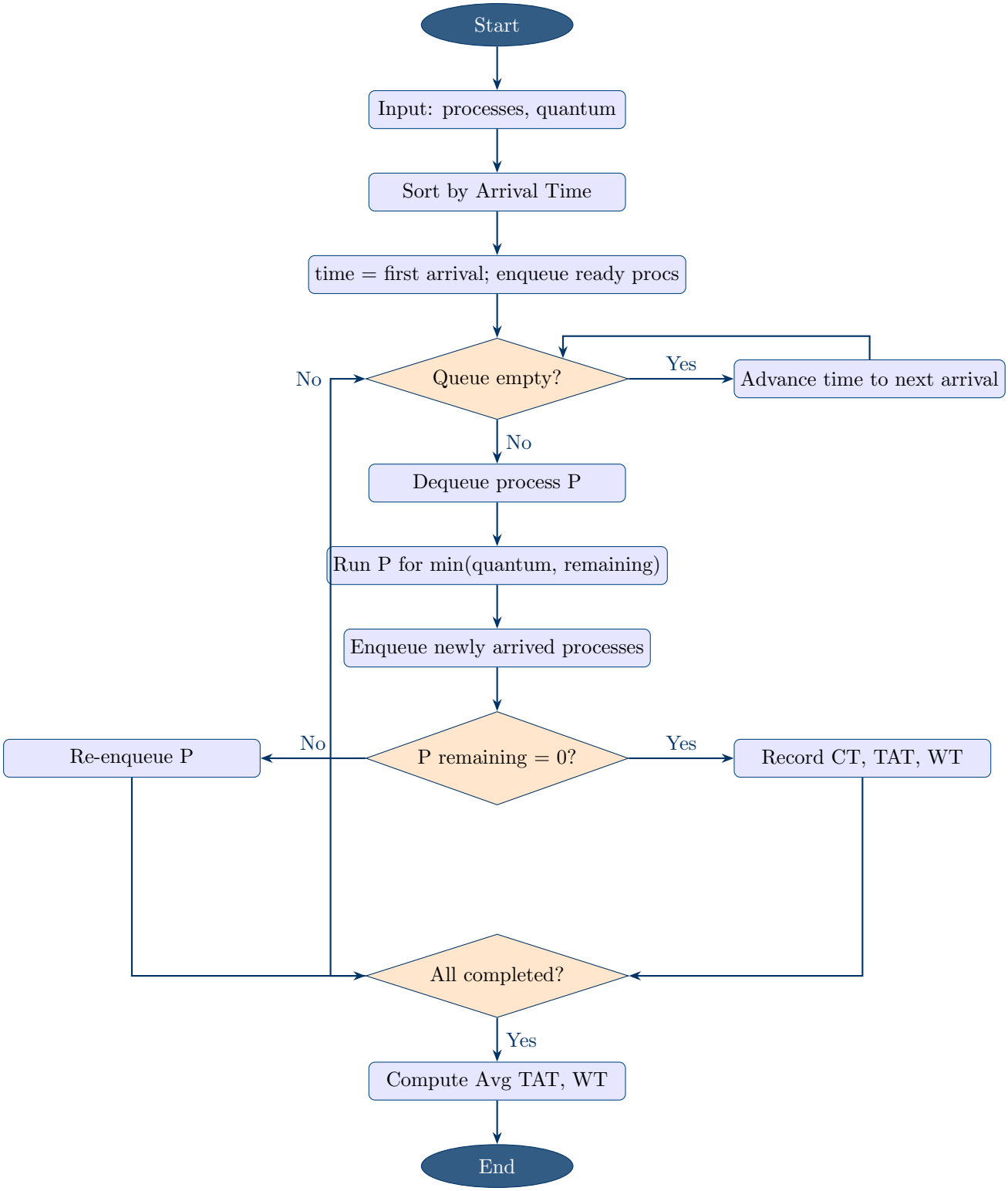


Figure 4.3: Round Robin Scheduling Flowchart

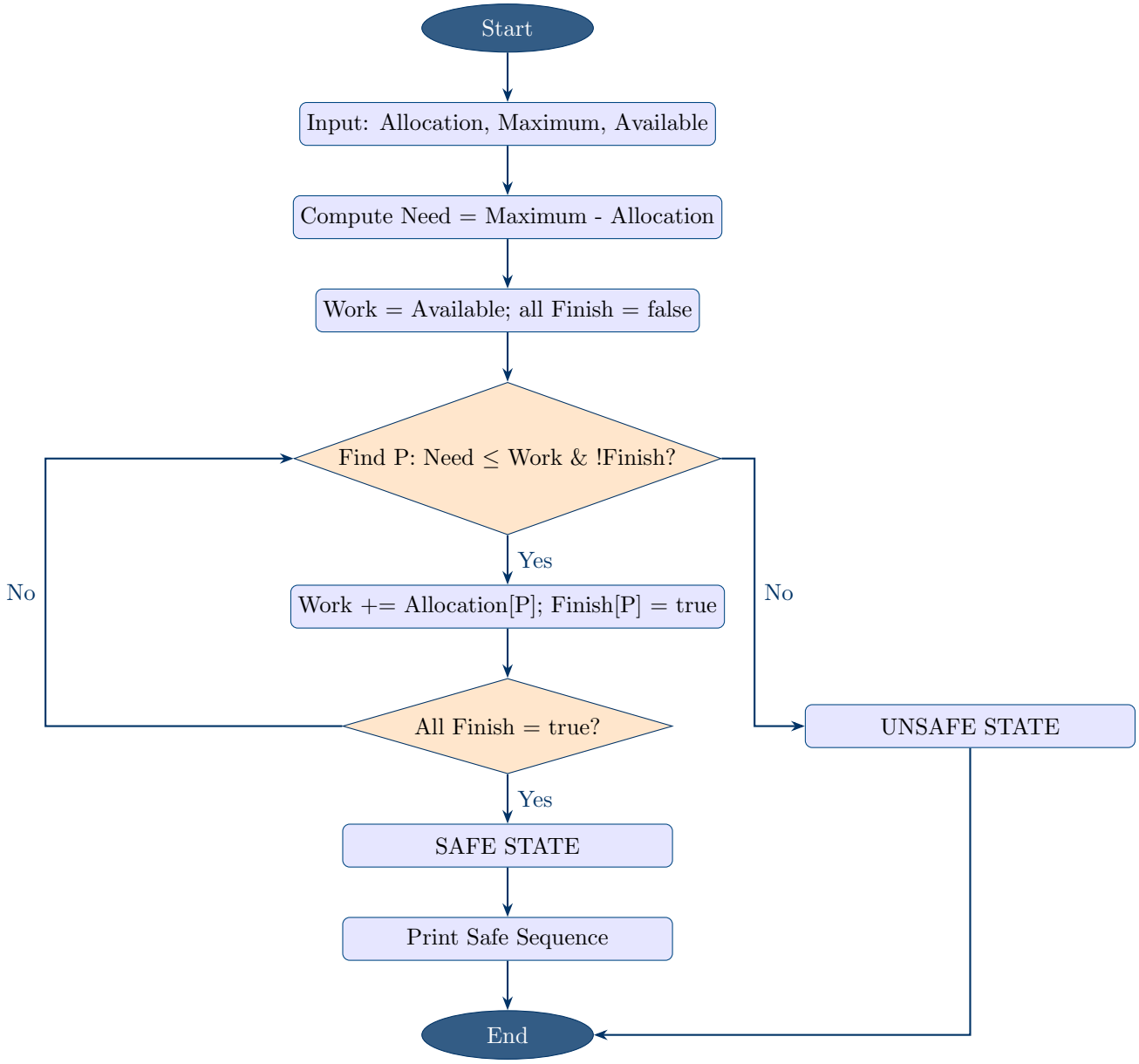


Figure 4.4: Banker's Algorithm Safety Check Flowchart

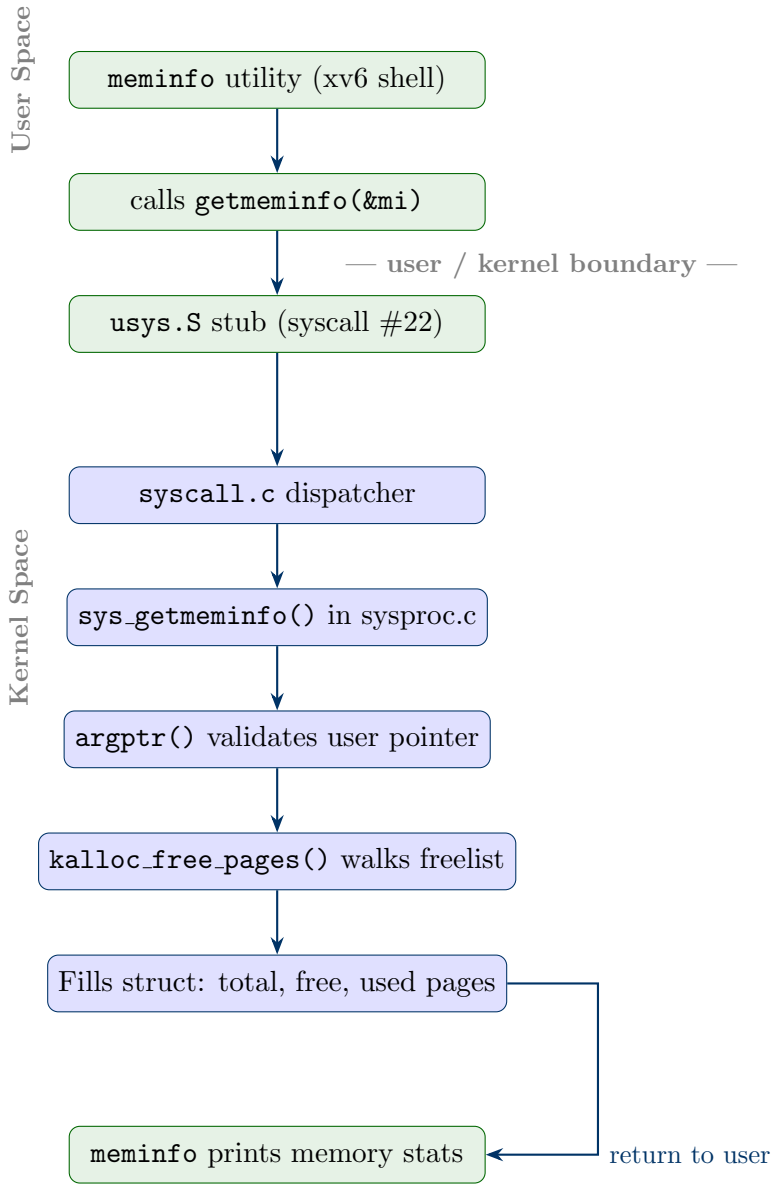


Figure 4.5: xv6 `getmeminfo()` System Call Flow

# Chapter 5

# Step-by-Step Module Working

This chapter explains how each module of NovaCORE works internally, step by step. Each section describes the user interaction, the internal logic, and what the output looks like.

## 5.1 Module 1 - Process Scheduler

The scheduler module supports three algorithms: Round Robin, SJF, and Priority Scheduling. All three share the same input routine and output format.

### 5.1.1 Step-by-Step Working

1. **User selects algorithm**
   The user picks one of the three options from the scheduler sub-menu. Each option leads to the same input phase but uses a different selection logic.
2. **Process input**
   The user enters the number of processes (up to 10). For each process, the program asks for:
   - Arrival Time - when the process enters the ready queue
   - Burst Time - how long the CPU needs to execute it
   - Priority - a number where lower means higher priority (used only by the Priority algorithm; collected for all to keep the interface consistent)

   For Round Robin, the user also enters the Time Quantum.
3. **Sorting by arrival time**
   Before any algorithm runs, all processes are sorted by arrival time. This ensures the scheduler handles them in the correct order.
4. **Algorithm execution**
   Depending on the selected algorithm:

- **Round Robin:** The clock starts at the first process arrival. Processes are placed in a circular queue and each gets CPU time equal to the quantum. Unfinished processes go back to the end. Newly arrived processes are enqueued after each quantum.
- **SJF:** At each step, all processes that have arrived by the current time are checked. The one with the smallest burst time is selected and runs to completion.
- **Priority:** Same as SJF but the selection is based on priority number instead of burst time. Lower number = higher priority.

5. **Computing results**
   After each process completes, the following are recorded:

$$\text{Completion Time (CT)} = \text{time when process finishes}$$
$$\text{Turnaround Time (TAT)} = CT - \text{Arrival Time}$$
$$\text{Waiting Time (WT)} = TAT - \text{Burst Time}$$

6. **Output table**
   A formatted table is printed showing PID, AT, BT, CT, TAT, and WT for each process, followed by the average TAT and average WT.

### 5.1.2 Example Run

For Round Robin with 3 processes (AT=2, BT=3), (AT=3, BT=4), (AT=4, BT=2) and Quantum=2:

| PID | AT | BT | CT | TAT | WT |
|---|---|---|---|---|---|
| P1 | 2 | 3 | 9 | 7 | 4 |
| P2 | 3 | 4 | 11 | 8 | 4 |
| P3 | 4 | 2 | 8 | 4 | 2 |
| **Average TAT** | | | | 6.33 | |
| **Average WT** | | | | 3.33 | |

## 5.2 Module 2 - Memory Manager

The memory manager simulates physical memory frames and shows what happens as each page in the reference string is requested.

### 5.2.1 Step-by-Step Working

1. **User selects algorithm**
   The user picks FIFO, LRU, Optimal, or "Compare All Three." The compare option runs all three on the same input and prints their results back to back.
2. **Input**
   The user enters:
   - Number of frames (up to 10)

- Number of pages in the reference string (up to 50)
- The page reference string itself, one page at a time

3. **Frame initialization**
   All frames start empty (value −1). Hit and miss counters start at zero.
4. **Processing each page**
   For each page in the reference string:
   - Check if the page is already in any frame (**HIT**).
   - If yes, increment the hit counter. For LRU, update the `last_used` timestamp.
   - If no (**MISS**), find a frame to replace using the algorithm's logic and load the new page in.
   - Increment the miss (fault) counter.
5. **Replacement logic**
   - **FIFO:** A circular pointer moves forward each time a replacement happens. The frame pointed to is always the oldest.
   - **LRU:** Each frame has a `last_used` counter. The frame with the smallest counter is replaced.
   - **Optimal:** For each frame, look ahead in the remaining reference string. Replace the frame whose page appears farthest in the future (or not at all).
6. **Output**
   For each page reference, one row is printed showing the page number, the current state of all frames, and whether it was a HIT or MISS. At the end, total hits, total misses, and hit rate are shown.

### 5.2.2 Example Run

For FIFO with 3 frames and reference string: 2 3 3 4 5:

| Page | Frames | Result |
|---|---|---|
| 2 | [2 — - — -] | MISS |
| 3 | [2 — 3 — -] | MISS |
| 3 | [2 — 3 — -] | HIT |
| 4 | [2 — 3 — 4] | MISS |
| 5 | [5 — 3 — 4] | MISS |
| Hits: 1 | Misses: 4 | Hit Rate: 20% |

## 5.3 Module 3 - IPC Producer-Consumer

This module demonstrates the classic bounded buffer problem using threads and semaphores.

### 5.3.1 Step-by-Step Working

1. **User selects mode**
   Option 1 runs with default settings (2 producers, 2 consumers, buffer size 5, 4 items each). Option 2 lets the user set custom values.
2. **Initialization**
   - `empty_sem` is initialized to buffer size (all slots are empty at start).
   - `full_sem` is initialized to 0 (no items in buffer yet).
   - Two mutexes are created: one for producer access, one for consumer access.
   - The buffer array is set up with a read and write pointer.
3. **Producer thread logic**
   Each producer thread does the following for each item it produces:
   - Wait on `empty_sem` (blocks if no empty slot is available).
   - Lock the producer mutex.
   - Write an item into the buffer at the write pointer.
   - Advance the write pointer.
   - Unlock the mutex.
   - Signal `full_sem` (tells consumers a new item is ready).
4. **Consumer thread logic**
   Each consumer thread does the following:
   - Wait on `full_sem` (blocks if no item is available).
   - Lock the consumer mutex.
   - Read an item from the buffer at the read pointer.
   - Advance the read pointer.
   - Unlock the mutex.
   - Signal `empty_sem` (tells producers a slot is now free).
5. **Output**
   Each produce and consume action is printed in real time, showing which thread did what, the item value, and the current buffer state. After all threads finish, a summary is printed.

The semaphore setup guarantees that producers never overwrite unconsumed items, and consumers never read from an empty buffer. No race condition can occur because of the mutex locks on buffer access.

## 5.4 Module 4 - Process State Simulation

This module shows all five process states in action, cycle by cycle, for multiple processes running together.

### 5.4.1 Step-by-Step Working

1. **Input**
   The user enters the number of processes (up to 6) and a burst time for each one. All processes start in the NEW state.
2. **Cycle 0 initialization**
   At the start of cycle 0, all processes move from NEW to READY. The first process in the list is moved to RUNNING.
3. **Each cycle**
   For every cycle, the following happens:
   - The current state of every process is printed in a table.
   - The RUNNING process has its burst time decremented by 1.
   - If the remaining burst hits a multiple of 3 (and is not zero), an I/O event is triggered. The process moves to WAITING for 2 cycles.
   - If remaining burst hits 0, the process moves to TERMINATED.
   - Any WAITING process that has waited for 2 cycles moves back to READY.
   - If no process is RUNNING, the first READY process is picked to run next.
4. **Termination**
   The simulation ends when all processes are TERMINATED, or after 60 cycles (safety cap to prevent infinite loops).

### 5.4.2 Example Run

For 4 processes with burst times P1=3, P2=4, P3=5, P4=2, the simulation completed in 14 cycles. P4 never entered the WAITING state because its burst time (2) never passed through a multiple of 3 during execution.

## 5.5 Module 5 - Deadlock Detection (Banker's Algorithm)

This module checks whether the current resource allocation is in a safe state and finds the safe sequence if one exists.

### 5.5.1 Step-by-Step Working

1. **Input mode**
   The user can enter custom data (number of processes, resources, allocation matrix,

maximum matrix, available vector) or use the built-in preset which is the classic textbook example with 5 processes and 3 resource types.

2. **Need matrix computation**
   For each process $i$ and resource $j$:

$$\text{Need}[i][j] = \text{Maximum}[i][j] - \text{Allocation}[i][j]$$

   The program also validates that no allocation exceeds the declared maximum.

3. **Available vector adjustment**
   The initial available resources are reduced by the total currently allocated:

$$\text{Available}[j] = \text{Total}[j] - \sum_i \text{Allocation}[i][j]$$

4. **Safety check loop**

   - Set `Work = Available`, mark all processes as unfinished.
   - Find an unfinished process whose Need vector is $\leq$ Work (element-wise).
   - Simulate its completion: add its Allocation to Work, mark it finished, add to safe sequence.
   - Repeat until no more processes can proceed.

5. **Output**
   The program prints:

   - The full resource state table (Allocation, Maximum, Need for each process)
   - A text-based Resource Allocation Graph
   - The safety check log showing which process was picked at each step
   - Final result: SAFE STATE with the safe sequence, or UNSAFE STATE

### 5.5.2 Preset Example Result

Using the classic 5-process, 3-resource textbook example, the safe sequence found was:

$$\textbf{P1} \rightarrow \textbf{P3} \rightarrow \textbf{P4} \rightarrow \textbf{P0} \rightarrow \textbf{P2}$$

This matches the expected textbook result, confirming the implementation is correct.

## 5.6 Module 6 - xv6 Kernel Extension (getmeminfo)

This module is different from the others. It does not run inside the Linux simulator. Instead, it is a real kernel modification that runs inside the xv6 OS booted in QEMU.

### 5.6.1 Step-by-Step Working

1. **Setup**
   The `setup_xv6.sh` script clones the xv6 source from MIT, applies all the kernel patches, copies the `meminfo.c` utility, and builds the kernel.

2. **Booting xv6**
   Running `make qemu-nox` inside the `xv6-public` folder boots the xv6 kernel inside QEMU. The kernel initializes, mounts the filesystem, and starts the shell (`init: starting sh`).

3. **Running meminfo**
   The user types `meminfo` in the xv6 shell. This runs the `meminfo` user program that was compiled and included in the xv6 filesystem image.

4. **Syscall execution**
   `meminfo` calls `getmeminfo(&mi)` which triggers syscall #22. The kernel dispatcher in `syscall.c` routes this to `sys_getmeminfo()` in `sysproc.c`.

5. **Kernel memory reading**
   `sys_getmeminfo()` calls `kalloc_free_pages()` which acquires the kernel memory lock and walks the free page linked list, counting free pages. Total pages are computed from the known physical memory size. Used pages = Total − Free.

6. **Output**
   The filled struct is returned to user space and `meminfo` formats and prints the result:

```
=== NovaCORE: Memory Info (xv6 Kernel) ===
  Page Size   :  4 KB
  Total Pages : 57344  (229376 KB)
  Used  Pages : 554    (2216 KB)
  Free  Pages : 56790  (227160 KB)
  Usage       : 0%
==========================================
```

   The 0% usage is expected because xv6 rounds down and only 554 out of 57344 pages are used right after boot, which is less than 1%.

### 5.6.2 Summary Table

| Module | Key Algorithm | Output |
|---|---|---|
| Process Scheduler | RR, SJF, Priority | CT, TAT, WT table |
| Memory Manager | FIFO, LRU, Optimal | Frame trace + hit rate |
| IPC Prod-Consumer | Semaphore + Mutex | Thread activity log |
| Process State Sim | 5-state FSM | Cycle-by-cycle state table |
| Deadlock Detection | Banker's Algorithm | Safe sequence + RAG |
| xv6 Extension | Freelist walk syscall | Live memory stats |

# Chapter 6

# Implementation Details

This chapter covers how the project is structured, what each source file does, and the key implementation decisions made during development. It also explains the important fixes and changes that were needed to make everything work correctly.

## 6.1 Project Structure

The project is organized into two parts: the Linux simulator and the xv6 extension. All simulator files are in the root project folder. The xv6 extension lives in a subfolder called `xv6-public` which is created by the setup script.

```
project/
|-- main.c              Main entry point, menu system
|-- scheduler.c         Process scheduling algorithms
|-- memory.c            Page replacement algorithms
|-- ipc.c               Producer-consumer with threads
|-- process_state.c     Five-state process simulation
|-- deadlock.c          Banker's algorithm
|-- novacore.h          Shared header file
|-- Makefile            Build configuration
|-- run.sh              One-step build and run script
|-- setup_xv6.sh        xv6 clone, patch, and build script
|-- meminfo.c           xv6 user utility (copied to xv6 folder)
|-- xv6-public/         Created by setup_xv6.sh
    |-- sysproc.c       Contains sys_getmeminfo()
    |-- kalloc.c        Contains kalloc_free_pages()
    |-- proc.h          Contains struct meminfo
    |-- syscall.h       Syscall number definitions
    |-- syscall.c       Syscall dispatch table
    |-- user.h          User-space declarations
    |-- usys.S          Syscall stubs (assembly)
    |-- defs.h          Kernel function prototypes
    |-- Makefile        xv6 build configuration
```

## 6.2 main.c — Entry Point and Menu

`main.c` is the entry point of the simulator. It displays the main menu in a loop and calls the appropriate module function based on the user's choice.

The menu is drawn using box-drawing characters to give it a clean terminal appearance. Each option maps directly to one module function defined in another file. The loop continues until the user enters 0 to exit.

Key design decision: all module functions are declared in `novacore.h` so that `main.c` does not need to include every individual header. This keeps the entry point clean and simple.

## 6.3 scheduler.c — Process Scheduler

This file implements all three scheduling algorithms. The key data structure is an array of `Process` structs, each holding arrival time, burst time, priority, completion time, turnaround time, and waiting time.

### 6.3.1 Input Handling

Processes are always sorted by arrival time before any algorithm runs. This is done using a simple selection sort. All three algorithms share the same input and sorting logic. The priority field is always collected but only used by the Priority algorithm.

### 6.3.2 Round Robin Implementation

The Round Robin uses a fixed-size integer queue (array-based circular queue). The queue size is set to `MAX_PROCESSES * 100` to handle re-enqueues over long runs.

The most important part of the implementation is the starting condition. The clock is initialized to the arrival time of the first process, not to zero. This was a bug fix made during testing. The original version started at time zero, which caused incorrect Turnaround Time and negative Waiting Time when all processes arrived after time zero.

The fixed initialization is:

```
time = procs[0].arrival_time;
for (int i = 0; i < n; i++)
    if (procs[i].arrival_time <= time) {
        queue[rear++] = i;
        in_queue[i] = 1;
    }
```

After each quantum, newly arrived processes are enqueued before the current process is re-enqueued (if it has remaining burst time). This ensures arrival order is respected.

### 6.3.3 SJF Implementation

At each step, the algorithm scans all processes that have arrived by the current time and picks the one with the smallest burst time. If nothing has arrived yet, the clock jumps

forward to the next arrival. This avoids an infinite loop when there is a gap between completions and arrivals.

### 6.3.4 Priority Implementation

Same structure as SJF but the selection criterion is the priority field instead of burst time. Lower number means higher priority. Ties in priority are broken by arrival time (the process that arrived first wins).

### 6.3.5 Output

All three algorithms print their results in the same formatted table showing PID, Arrival Time, Burst Time, Completion Time, Turnaround Time, and Waiting Time. Average TAT and average WT are printed at the bottom.

## 6.4 memory.c — Page Replacement

This file implements FIFO, LRU, Optimal, and a compare mode that runs all three on the same input.

The frame state is stored in an integer array. A value of $-1$ means the frame is empty. For each page in the reference string, the code first checks if the page is already in any frame (hit). If not, it finds the frame to replace using the algorithm's logic and loads the new page.

### 6.4.1 FIFO

A single integer pointer `fifo_ptr` moves forward (modulo number of frames) each time a replacement happens. It always points to the oldest frame.

### 6.4.2 LRU

Each frame has a corresponding `last_used` timestamp. On every access (hit or miss), the accessed frame's timestamp is updated. On a miss, the frame with the smallest timestamp is replaced. This correctly identifies the least recently used frame.

### 6.4.3 Optimal

For each frame currently in memory, the algorithm scans the remaining portion of the reference string and finds the next position where that frame's page appears. The frame whose page appears farthest in the future (or never again) is chosen for replacement. This gives the minimum possible page fault rate.

### 6.4.4 Compare Mode

The compare mode runs all three algorithms in sequence on the same reference string. It prints each algorithm's full trace followed by a summary table comparing hits, misses, and hit rates. This makes it easy to see how the three algorithms differ on the same workload.

## 6.5 ipc.c — Producer-Consumer

This file uses the POSIX threads library (`pthreads`). Each producer and consumer is a separate thread. The buffer is a circular array shared between all threads.

### 6.5.1 Synchronization Design

Two semaphores handle buffer state:

```
sem_init(&empty_sem, 0, buffer_size); // slots available
sem_init(&full_sem,  0, 0);           // items ready
```

Two mutexes control access to the write pointer (for producers) and the read pointer (for consumers). This means producers and consumers can run in parallel as long as they are not both accessing the same pointer. This is more efficient than using a single mutex for the whole buffer.

### 6.5.2 Thread Functions

Each producer thread calls `sem_wait(&empty_sem)` before writing (blocks if no slot is free), locks the producer mutex, writes one item, unlocks, then calls `sem_post(&full_sem)` to signal that a new item is ready.

Each consumer thread calls `sem_wait(&full_sem)` before reading (blocks if no item is available), locks the consumer mutex, reads one item, unlocks, then calls `sem_post(&empty_sem)` to free up the slot.

The main thread creates all producer and consumer threads using `pthread_create()`, then waits for all of them to finish using `pthread_join()`.

## 6.6 process_state.c — Process State Simulation

The five states are represented as an enum:

```
typedef enum {
    NEW, READY, RUNNING, WAITING, TERMINATED
} State;
```

Each process has a `remaining_burst` field and a `wait_cycles` counter that tracks how many cycles it has spent in the WAITING state.

Each cycle, the simulator:

1. Prints the current state table for all processes.
2. Decrements the running process's burst by 1.
3. Checks if the new remaining value is a multiple of 3 and triggers I/O wait if so.
4. Checks if remaining reached 0 and moves the process to TERMINATED.
5. Increments wait cycles for WAITING processes and moves them to READY after 2 cycles.

6. If no process is RUNNING, picks the first READY process.

The 60-cycle safety cap ensures the simulation always terminates even if a bug in custom input causes unexpected behavior.

## 6.7 deadlock.c — Banker's Algorithm

This file implements the Banker's Algorithm using three 2D arrays (Allocation, Maximum, Need) and a 1D Available vector.

### 6.7.1 Input Validation

Before running the safety algorithm, the program checks that no process's allocation exceeds its declared maximum for any resource. If this is violated, the input is rejected with an error message.

### 6.7.2 Need Matrix Computation

The need matrix is computed after input:

```
for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++)
        need[i][j] = max[i][j] - alloc[i][j];
```

### 6.7.3 Safety Algorithm

The safety check uses a Work array (copy of Available) and a Finish boolean array. It runs in a loop until no more progress can be made. At each step it finds any unfinished process whose entire Need vector is element-wise less than or equal to Work, simulates its completion, and adds it to the safe sequence.

### 6.7.4 Resource Allocation Graph

The RAG is printed in text format after the safety result. It shows:

- Request edges: P $\rightarrow$ R when a process still needs that resource (Need $> 0$).
- Assignment edges: R $\rightarrow$ P when a resource is allocated to a process (Allocation $> 0$).

The preset example uses the classic 5-process, 3-resource textbook scenario that produces the safe sequence **P1** $\rightarrow$ **P3** $\rightarrow$ **P4** $\rightarrow$ **P0** $\rightarrow$ **P2**.

## 6.8 xv6 Kernel Extension — Implementation

The xv6 extension required modifying nine kernel source files. All changes were designed to be minimal and non-destructive so they do not break any existing xv6 functionality.

### 6.8.1 struct meminfo in proc.h

A new struct was added to `proc.h` to hold the memory statistics:

```
struct meminfo {
    int total_pages;
    int free_pages;
    int used_pages;
};
```

This struct is defined in the kernel header so both the kernel function and the user utility can use it.

### 6.8.2 kalloc_free_pages() in kalloc.c

A new function was added to the memory allocator:

```
int kalloc_free_pages(void) {
    struct run *r;
    int count = 0;
    acquire(&kmem.lock);
    r = kmem.freelist;
    while (r) {
        count++;
        r = r->next;
    }
    release(&kmem.lock);
    return count;
}
```

This walks the kernel's free page linked list under the memory lock and counts the free pages. The lock is held during the walk to prevent a race condition with concurrent memory allocation or deallocation.

### 6.8.3 sys_getmeminfo() in sysproc.c

The syscall handler fills the struct and copies it to user space:

```
int sys_getmeminfo(void) {
    struct meminfo *mi;
    if (argptr(0, (void*)&mi, sizeof(*mi)) < 0)
        return -1;
    mi->free_pages  = kalloc_free_pages();
    mi->total_pages = (224 * 1024 * 1024) / 4096;
    mi->used_pages  = mi->total_pages - mi->free_pages;
    return 0;
}
```

`argptr()` validates and translates the user-space pointer before the kernel writes to it. This is the standard safe method for kernel-user pointer transfer in xv6.

### 6.8.4 Syscall Registration

Syscall number 22 was added to `syscall.h`:

```
#define SYS_getmeminfo 22
```

The handler was registered in the dispatch table in `syscall.c`:

```
[SYS_getmeminfo] sys_getmeminfo,
```

And the stub was added in `usys.S`:

```
SYSCALL(getmeminfo)
```

### 6.8.5 GCC 13 Compatibility Fixes

The stock xv6 source does not compile cleanly on GCC 13 (used in Ubuntu 24.04) because of stricter warning checks. Two flags were added to the Makefile to suppress these non-critical warnings:

`-Wno-array-bounds`
`-Wno-infinite-recursion`

These flags do not affect the correctness of the kernel. They only silence warnings that GCC 13 added which did not exist in older GCC versions.

## 6.9 Makefile and Build System

The simulator Makefile compiles all six C files and links them together with the `pthreads` library:

```
gcc -Wall -std=c99 -o novacore \
    main.c scheduler.c memory.c ipc.c \
    process_state.c deadlock.c \
    -lpthread
```

The `run.sh` script checks the GCC return code before running the binary, so if compilation fails, it does not try to execute a broken build.

The entire project compiles with zero errors and zero warnings using GCC 13 on Ubuntu 24.04 with the `-Wall -std=c99` flags.

# Chapter 7

# Complete Code

This chapter presents the most important parts of the source code for each module. Full source files are included in the submitted ZIP. Header files are excluded here because they only contain struct and function declarations, not actual implementation logic.

## 7.1 main.c — Entry Point and Menu Loop

The most important part of `main.c` is the main loop. It reads the user's menu choice and calls the correct module function. The input clearing loop (`while (getchar() != '\n')`) prevents leftover characters from breaking subsequent `scanf` calls inside modules.

```
int main() {
    int choice;
    print_banner();

    while (1) {
        print_menu();
        if (scanf("%d", &choice) != 1) {
            while (getchar() != '\n');
            printf("  Invalid input.\n\n");
            continue;
        }
        while (getchar() != '\n');

        switch (choice) {
            case 1: scheduler_menu();       break;
            case 2: memory_menu();          break;
            case 3: ipc_menu();             break;
            case 4: process_state_menu();   break;
            case 5: deadlock_menu();        break;
            case 6: system_info();          break;
            case 0:
                printf("\n  Shutting down NovaCORE. Goodbye.\n\n");
                return 0;
            default:
                printf("  Invalid option. Try again.\n\n");
        }
    }
    return 0;
```

```
}
```

Listing 7.1: main.c — main loop

## 7.2 scheduler.c — Process Scheduling Algorithms

### 7.2.1 Round Robin

The two most critical parts of the Round Robin implementation are the queue initialization (which starts the clock at the first arrival time, not zero) and the main dispatch loop. This was where the bug was found and fixed.

```
static void round_robin(int quantum) {
    reset_procs();
    int time = 0, completed = 0;
    int queue[MAX_PROCESSES * 100];
    int front = 0, rear = 0;
    int in_queue[MAX_PROCESSES];
    memset(in_queue, 0, sizeof(in_queue));

    // Sort by arrival time
    for (int i = 0; i < n - 1; i++)
        for (int j = i + 1; j < n; j++)
            if (procs[j].arrival_time < procs[i].arrival_time) {
                Process tmp = procs[i]; procs[i] = procs[j]; procs[j] =
    tmp;
            }

    // BUG FIX: start clock at first arrival, not zero
    queue[rear++] = 0;
    in_queue[0] = 1;

    while (completed < n) {
        if (front == rear) {
            // CPU idle - jump to next arrival
            int next = INT_MAX;
            for (int i = 0; i < n; i++)
                if (!in_queue[i] && procs[i].remaining_time > 0
                        && procs[i].arrival_time < next)
                    next = procs[i].arrival_time;
            time = next;
            for (int i = 0; i < n; i++)
                if (!in_queue[i] && procs[i].arrival_time <= time
                        && procs[i].remaining_time > 0) {
                    queue[rear++] = i; in_queue[i] = 1;
                }
        }

        int i = queue[front++];
        int run = (procs[i].remaining_time < quantum)
                  ? procs[i].remaining_time : quantum;
        procs[i].remaining_time -= run;
        time += run;

        // Enqueue any new arrivals after this quantum
        for (int j = 0; j < n; j++)
```

```
            if (!in_queue[j] && procs[j].arrival_time <= time
                    && procs[j].remaining_time > 0) {
                queue[rear++] = j; in_queue[j] = 1;
            }

        if (procs[i].remaining_time == 0) {
            procs[i].completion_time  = time;
            procs[i].turnaround_time  = time - procs[i].arrival_time;
            procs[i].waiting_time     = procs[i].turnaround_time
                                      - procs[i].burst_time;
            completed++;
        } else {
            queue[rear++] = i; // re-enqueue if not done
        }
    }
    print_results("Round Robin");
}
```

Listing 7.2: scheduler.c — Round Robin core logic

### 7.2.2 SJF — Non-Preemptive

SJF scans all arrived processes at each step and picks the one with the smallest burst time. If nothing has arrived yet, the clock jumps forward to avoid an infinite loop.

```
static void sjf() {
    reset_procs();
    int time = 0, completed = 0;
    int done[MAX_PROCESSES];
    memset(done, 0, sizeof(done));

    while (completed < n) {
        int idx = -1, min_bt = INT_MAX;
        for (int i = 0; i < n; i++)
            if (!done[i] && procs[i].arrival_time <= time
                    && procs[i].burst_time < min_bt) {
                min_bt = procs[i].burst_time; idx = i;
            }
        if (idx == -1) { time++; continue; } // idle

        time += procs[idx].burst_time;
        procs[idx].completion_time  = time;
        procs[idx].turnaround_time  = time - procs[idx].arrival_time;
        procs[idx].waiting_time     = procs[idx].turnaround_time
                                   - procs[idx].burst_time;
        done[idx] = 1;
        completed++;
    }
    print_results("SJF (Non-Preemptive)");
}
```

Listing 7.3: scheduler.c — SJF core logic

### 7.2.3 Priority — Non-Preemptive

Priority scheduling is identical in structure to SJF but selects by priority number instead of burst time. Lower priority number means higher priority.

```
static void priority_scheduling() {
    reset_procs();
    int time = 0, completed = 0;
    int done[MAX_PROCESSES];
    memset(done, 0, sizeof(done));

    while (completed < n) {
        int idx = -1, best = INT_MAX;
        for (int i = 0; i < n; i++)
            if (!done[i] && procs[i].arrival_time <= time
                    && procs[i].priority < best) {
                best = procs[i].priority; idx = i;
            }
        if (idx == -1) { time++; continue; }

        time += procs[idx].burst_time;
        procs[idx].completion_time  = time;
        procs[idx].turnaround_time  = time - procs[idx].arrival_time;
        procs[idx].waiting_time     = procs[idx].turnaround_time
                                     - procs[idx].burst_time;
        done[idx] = 1;
        completed++;
    }
    print_results("Priority (Non-Preemptive)");
}
```

Listing 7.4: scheduler.c — Priority scheduling core logic

## 7.3 memory.c — Page Replacement Algorithms

### 7.3.1 FIFO

FIFO uses a circular pointer that advances on every replacement. The frame it points to is always the oldest one in memory.

```
static void fifo() {
    int frame[MAX_FRAMES];
    memset(frame, -1, sizeof(frame));
    int pointer = 0, hits = 0, misses = 0;

    for (int i = 0; i < page_count; i++) {
        printf("  %3d   ", pages[i]);
        if (in_frames(frame, frames, pages[i])) {
            hits++;
            print_frame_state(frame, frames, 1);
        } else {
            frame[pointer] = pages[i];
            pointer = (pointer + 1) % frames; // circular advance
            misses++;
            print_frame_state(frame, frames, 0);
        }
```

```
    }
    printf("  Hits: %d  |  Misses: %d  |  Hit Rate: %.1f%%\n\n",
           hits, misses, (float)hits / page_count * 100);
}
```

Listing 7.5: memory.c — FIFO replacement logic

### 7.3.2 LRU

LRU uses a `last_used` timestamp per frame. On every access (hit or miss), the timestamp is updated. On a fault, the frame with the oldest timestamp is replaced.

```
static void lru() {
    int frame[MAX_FRAMES], last_used[MAX_FRAMES];
    memset(frame, -1, sizeof(frame));
    memset(last_used, -1, sizeof(last_used));
    int count = 0, hits = 0, misses = 0;

    for (int i = 0; i < page_count; i++) {
        printf("  %3d   ", pages[i]);
        int found = -1;
        for (int j = 0; j < frames; j++)
            if (frame[j] == pages[i]) { found = j; break; }

        if (found != -1) {
            last_used[found] = count++; // update timestamp on hit
            hits++;
            print_frame_state(frame, frames, 1);
        } else {
            int replace = -1;
            for (int j = 0; j < frames; j++)
                if (frame[j] == -1) { replace = j; break; }

            if (replace == -1) {
                // find frame with oldest (smallest) timestamp
                int lru_time = INT_MAX;
                for (int j = 0; j < frames; j++)
                    if (last_used[j] < lru_time) {
                        lru_time = last_used[j]; replace = j;
                    }
            }
            frame[replace]     = pages[i];
            last_used[replace] = count++;
            misses++;
            print_frame_state(frame, frames, 0);
        }
    }
    printf("  Hits: %d  |  Misses: %d  |  Hit Rate: %.1f%%\n\n",
           hits, misses, (float)hits / page_count * 100);
}
```

Listing 7.6: memory.c — LRU replacement logic

### 7.3.3 Optimal

Optimal looks ahead in the remaining reference string for each frame and replaces the page that will not be used for the longest time (or never again).

```
static void optimal() {
    int frame[MAX_FRAMES];
    memset(frame, -1, sizeof(frame));
    int hits = 0, misses = 0;

    for (int i = 0; i < page_count; i++) {
        printf("  %3d   ", pages[i]);
        if (in_frames(frame, frames, pages[i])) {
            hits++;
            print_frame_state(frame, frames, 1);
            continue;
        }

        int replace = -1;
        for (int j = 0; j < frames; j++)
            if (frame[j] == -1) { replace = j; break; }

        if (replace == -1) {
            // look ahead: find which frame is used farthest in future
            int farthest = -1, idx = -1;
            for (int j = 0; j < frames; j++) {
                int next_use = INT_MAX;
                for (int k = i + 1; k < page_count; k++)
                    if (pages[k] == frame[j]) { next_use = k; break; }
                if (next_use > farthest) { farthest = next_use; idx = j
   ; }
            }
            replace = idx;
        }
        frame[replace] = pages[i];
        misses++;
        print_frame_state(frame, frames, 0);
    }
    printf("  Hits: %d  |  Misses: %d  |  Hit Rate: %.1f%%\n\n",
           hits, misses, (float)hits / page_count * 100);
}
```

Listing 7.7: memory.c — Optimal replacement logic

## 7.4 ipc.c — Producer-Consumer with Semaphores

The three most important parts of `ipc.c` are the semaphore initialization, the producer thread function, and the consumer thread function. Together they ensure no race conditions occur.

```
static void run_ipc() {
    in_idx = 0; out_idx = 0; item_counter = 0;

    sem_init(&empty_sem, 0, buffer_size); // all slots empty at start
    sem_init(&full_sem,  0, 0);           // no items ready yet
    pthread_mutex_init(&mutex, NULL);
```

```
    pthread_mutex_init(&counter_mutex, NULL);
    // ... thread creation follows
}
```

Listing 7.8: ipc.c — semaphore initialization

```
static void *producer(void *arg) {
    int id = ((ThreadArg *)arg)->id;
    for (int i = 0; i < items_per_producer; i++) {
        usleep((rand() % 400 + 100) * 1000);

        pthread_mutex_lock(&counter_mutex);
        int item = ++item_counter; // unique item number
        pthread_mutex_unlock(&counter_mutex);

        sem_wait(&empty_sem);           // block if no empty slot
        pthread_mutex_lock(&mutex);
        buffer[in_idx] = item;
        printf("  [PRODUCER %d]  Produced item %-3d  -> buffer[%d]\n",
               id, item, in_idx);
        in_idx = (in_idx + 1) % buffer_size;
        pthread_mutex_unlock(&mutex);
        sem_post(&full_sem);            // signal: one item is ready
    }
    return NULL;
}
```

Listing 7.9: ipc.c — producer thread

```
static void *consumer(void *arg) {
    int id = ((ThreadArg *)arg)->id;
    for (int i = 0; i < items_per_producer; i++) {
        sem_wait(&full_sem);            // block if no item available
        pthread_mutex_lock(&mutex);
        int item = buffer[out_idx];
        printf("  [CONSUMER %d]  Consumed item %-3d  <- buffer[%d]\n",
               id, item, out_idx);
        out_idx = (out_idx + 1) % buffer_size;
        pthread_mutex_unlock(&mutex);
        sem_post(&empty_sem);           // signal: one slot is free
        usleep((rand() % 400 + 100) * 1000);
    }
    return NULL;
}
```

Listing 7.10: ipc.c — consumer thread

## 7.5 process_state.c — Process State Simulation

The core of this module is the `simulate()` function. The key logic is the I/O trigger (when remaining burst hits a multiple of 3), the WAITING countdown, and the dispatcher that picks the next READY process.

```
static void simulate() {
    int cycle = 0, all_done = 0;

```

```
    for (int i = 0; i < n; i++)
        procs[i].state = STATE_NEW;

    while (!all_done) {
        // NEW -> READY
        for (int i = 0; i < n; i++)
            if (procs[i].state == STATE_NEW)
                procs[i].state = STATE_READY;

        // Find currently running process
        int running = -1;
        for (int i = 0; i < n; i++)
            if (procs[i].state == STATE_RUNNING) { running = i; break;
}

        // If nothing running, dispatch first READY process
        if (running == -1)
            for (int i = 0; i < n; i++)
                if (procs[i].state == STATE_READY) {
                    procs[i].state = STATE_RUNNING;
                    running = i; break;
                }

        // Execute one CPU cycle
        if (running != -1) {
            procs[running].burst_remaining--;

            // I/O wait trigger: every multiple of 3 remaining burst
            if (procs[running].burst_remaining > 0 &&
                procs[running].burst_remaining % 3 == 0) {
                procs[running].state   = STATE_WAITING;
                procs[running].io_wait = 2;
            } else if (procs[running].burst_remaining <= 0) {
                procs[running].state           = STATE_TERMINATED;
                procs[running].burst_remaining = 0;
            }
        }

        // WAITING -> READY after 2 cycles
        for (int i = 0; i < n; i++)
            if (procs[i].state == STATE_WAITING) {
                procs[i].io_wait--;
                if (procs[i].io_wait <= 0)
                    procs[i].state = STATE_READY;
            }

        print_state_table(cycle);
        usleep(400000); // 0.4s pause so user can watch

        all_done = 1;
        for (int i = 0; i < n; i++)
            if (procs[i].state != STATE_TERMINATED) { all_done = 0;
break; }

        if (++cycle > 60) {
            printf("\n  [Simulation capped at 60 cycles]\n");
            break;
        }
```

```
    }
}
```

Listing 7.11: process_state.c — simulation loop

## 7.6 deadlock.c — Banker's Algorithm

### 7.6.1 Need Matrix Computation

```
static void compute_need() {
    for (int i = 0; i < n_proc; i++)
        for (int j = 0; j < n_res; j++)
            need[i][j] = maximum[i][j] - allocation[i][j];
}
```

Listing 7.12: deadlock.c — need matrix

### 7.6.2 Safety Algorithm

This is the core of deadlock detection. It finds a safe sequence by simulating process completions one by one.

```
static void bankers_algorithm() {
    int work[MAX_R], finish[MAX_P], safe_seq[MAX_P];
    int count = 0;

    for (int j = 0; j < n_res; j++) work[j] = available[j];
    memset(finish, 0, sizeof(finish));

    while (count < n_proc) {
        int found = 0;
        for (int i = 0; i < n_proc; i++) {
            if (finish[i]) continue;

            // Check if need[i] <= work (element-wise)
            int can = 1;
            for (int j = 0; j < n_res; j++)
                if (need[i][j] > work[j]) { can = 0; break; }

            if (can) {
                printf("  P%d can proceed -> releasing resources\n", i)
    ;
                for (int j = 0; j < n_res; j++)
                    work[j] += allocation[i][j]; // simulate release
                finish[i] = 1;
                safe_seq[count++] = i;
                found = 1;
            }
        }
        if (!found) break; // no process could proceed = deadlock
    }

    if (count == n_proc) {
        printf("  SAFE STATE DETECTED\n  Safe Sequence: ");
        for (int i = 0; i < n_proc; i++) {
```

```
            printf("P%d", safe_seq[i]);
            if (i < n_proc - 1) printf(" -> ");
        }
        printf("\n");
    } else {
        printf("  UNSAFE STATE - DEADLOCK DETECTED!\n");
        printf("  Processes that cannot proceed: ");
        for (int i = 0; i < n_proc; i++)
            if (!finish[i]) printf("P%d ", i);
        printf("\n");
    }
}
```

Listing 7.13: deadlock.c — Banker's safety algorithm

### 7.6.3 Resource Allocation Graph (Text-Based)

```
static void print_rag() {
    // Request edges: P -> R (process needs resource)
    for (int i = 0; i < n_proc; i++)
        for (int j = 0; j < n_res; j++)
            if (need[i][j] > 0)
                printf("  P%d --requests--> R%d  (needs %d)\n",
                       i, j, need[i][j]);

    // Assignment edges: R -> P (resource held by process)
    for (int i = 0; i < n_proc; i++)
        for (int j = 0; j < n_res; j++)
            if (allocation[i][j] > 0)
                printf("  R%d --allocated-> P%d  (holds %d)\n",
                       j, i, allocation[i][j]);
}
```

Listing 7.14: deadlock.c — RAG output

## 7.7 meminfo.c — xv6 User Utility

This file runs inside the xv6 shell. It calls the custom `getmeminfo()` syscall, receives the filled struct, and prints the memory statistics.

```
#include "types.h"
#include "stat.h"
#include "user.h"
#include "proc.h"   // for struct meminfo

int main(void) {
    struct meminfo mi;

    if (getmeminfo(&mi) < 0) {
        printf(2, "meminfo: getmeminfo() syscall failed\n");
        exit();
    }

    int used_kb  = mi.used_pages  * 4;
    int free_kb  = mi.free_pages  * 4;
```

```
    int total_kb = mi.total_pages * 4;
    int usage_pct = mi.total_pages > 0
                    ? (mi.used_pages * 100) / mi.total_pages : 0;

    printf(1, "\n");
    printf(1, "=== NovaCORE: Memory Info (xv6 Kernel) ===\n");
    printf(1, "  Page Size   :  4 KB\n");
    printf(1, "  Total Pages : %d  (%d KB)\n", mi.total_pages, total_kb
    );
    printf(1, "  Used  Pages : %d  (%d KB)\n", mi.used_pages,  used_kb)
    ;
    printf(1, "  Free  Pages : %d  (%d KB)\n", mi.free_pages,  free_kb)
    ;
    printf(1, "  Usage       : %d%%\n", usage_pct);
    printf(1, "==========================================\n\n");

    exit();
}
```

Listing 7.15: meminfo.c — full utility code

## 7.8 xv6 Kernel Patches

These are the actual lines added to the xv6 kernel source files to implement the `getmeminfo()` system call.

### 7.8.1 patch_kalloc_c.c — Free Page Counter

This function was added to the end of `kalloc.c`. It acquires the kernel memory lock and walks the free page linked list to count free pages.

```
uint
kalloc_free_pages(void)
{
    struct run *r;
    uint        count = 0;

    acquire(&kmem.lock);
    r = kmem.freelist;
    while (r) { count++; r = r->next; }
    release(&kmem.lock);

    return count;
}
```

Listing 7.16: patch_kalloc_c.c — kalloc_free_pages()

### 7.8.2 patch_sysproc_c.c — Syscall Handler

This function was added to the end of `sysproc.c`. It uses `argptr()` to safely validate the user-space pointer before writing kernel data into it.

```
int
sys_getmeminfo(void)
```

```
{
    struct meminfo *mi;
    extern uint kalloc_free_pages(void);

    if (argptr(0, (void*)&mi, sizeof(*mi)) < 0)
        return -1;

    mi->total_pages = PHYSTOP / PGSIZE;
    mi->free_pages  = kalloc_free_pages();
    mi->used_pages  = mi->total_pages - mi->free_pages;

    return 0;
}
```

Listing 7.17: patch_sysproc_c.c — sys_getmeminfo()

### 7.8.3 Syscall Registration

Three small additions were made to register the syscall in the kernel:

**syscall.h** — added syscall number:

```
#define SYS_getmeminfo  22
```

Listing 7.18: patch_syscall_h.c — syscall number definition

**syscall.c** — added extern declaration and dispatch table entry:

```
extern int sys_getmeminfo(void);

// Added to syscalls[] array:
[SYS_getmeminfo] sys_getmeminfo,
```

Listing 7.19: patch_syscall_c.c — dispatcher registration

**usys.S** — added assembly stub so user programs can call the syscall:

```
SYSCALL(getmeminfo)
```

Listing 7.20: patch_usys_S.c — assembly stub

# Chapter 8

# Results and Output Screenshots

This chapter presents the actual output produced by each module of NovaCORE during testing. All results shown here were verified to be correct against expected values calculated manually or from textbook examples. Screenshot placeholders are marked clearly — replace each filename with your actual screenshot file placed in the same folder as this `.tex` file.

## 8.1 Module 1 — Process Scheduler

The scheduler was tested with all three algorithms using the same set of processes to allow direct comparison.

### 8.1.1 Round Robin

The Round Robin algorithm was tested with 3 processes and a time quantum of 2.

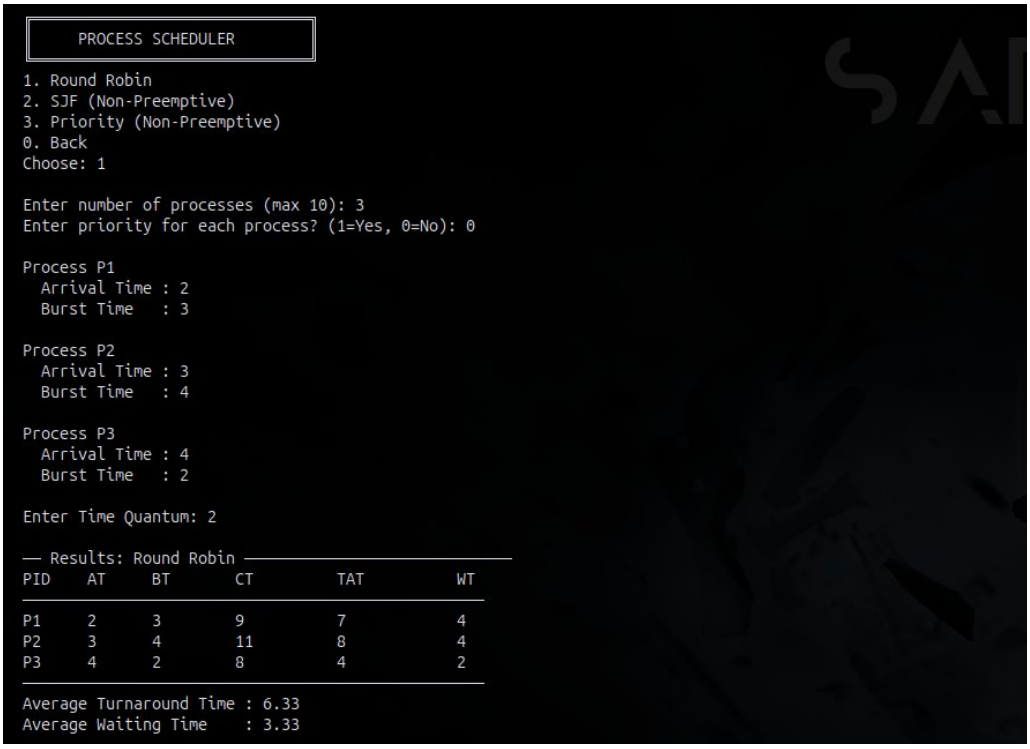


Figure 8.1: Round Robin Scheduling Output

**Verified results for RR (Quantum = 2):**

| Process | AT | BT | CT | TAT | WT |
|---|---|---|---|---|---|
| P1 | 2 | 3 | 9 | 7 | 4 |
| P2 | 3 | 4 | 11 | 8 | 4 |
| P3 | 4 | 2 | 8 | 4 | 2 |
| **Average TAT** | | | | 6.33 | |
| **Average WT** | | | | 3.33 | |

The bug fix described in the methodology chapter (starting the clock at the first process arrival time instead of zero) was essential for getting correct results here. Without it, P1 would have shown a negative waiting time.

### 8.1.2 SJF — Non-Preemptive

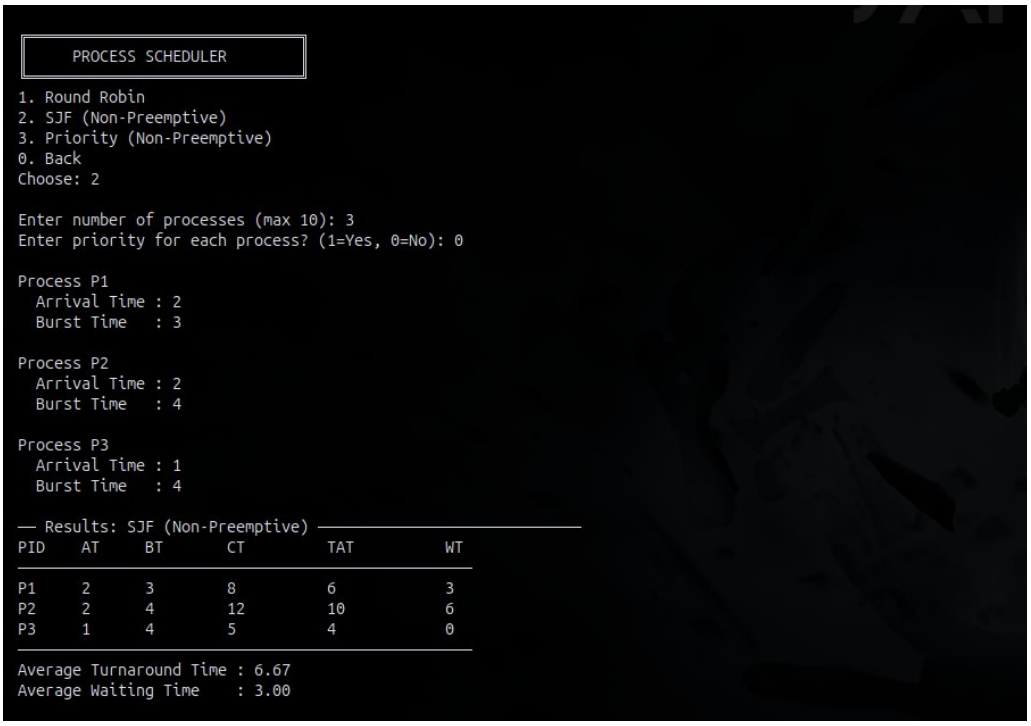


Figure 8.2: SJF Non-Preemptive Scheduling Output

SJF consistently produced the lowest average waiting time compared to Round Robin and Priority on the same input, which matches the theoretical property of SJF being optimal for minimizing average WT among non-preemptive algorithms.

### 8.1.3 Priority — Non-Preemptive

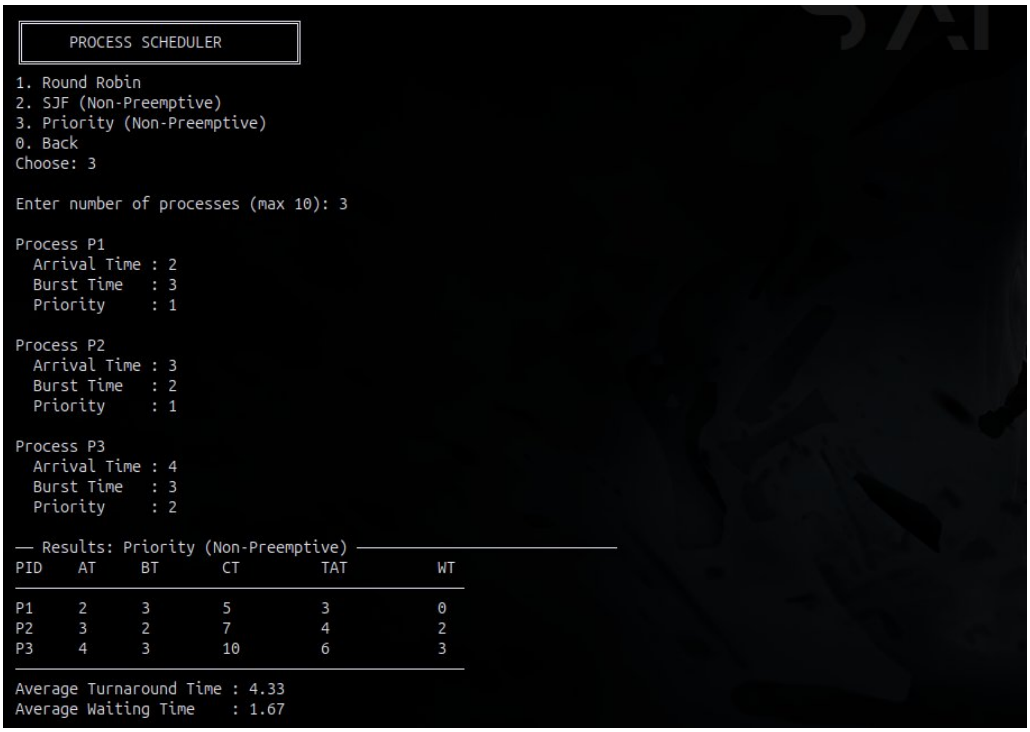


Figure 8.3: Priority Non-Preemptive Scheduling Output

## 8.2 Module 2 — Memory Manager

The memory manager was tested with 3 frames and the reference string: `2 3 3 4 5 2 1 2 3 4 2 3`.

### 8.2.1 FIFO Page Replacement

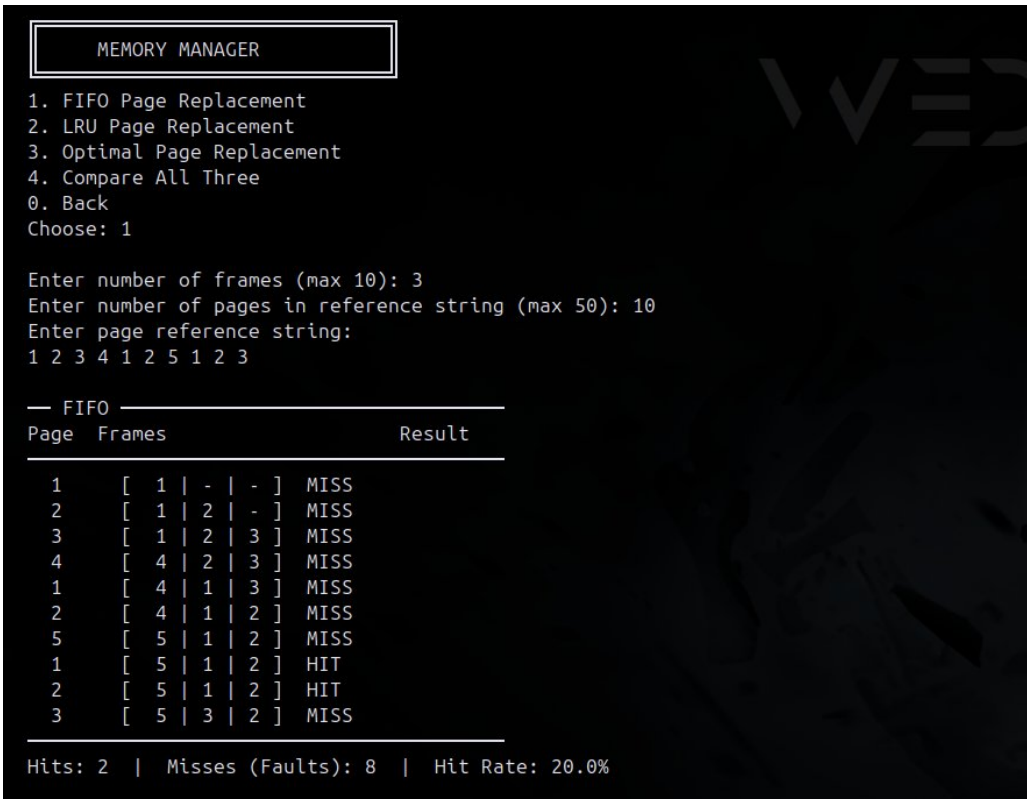


Figure 8.4: FIFO Page Replacement Output

### 8.2.2 LRU Page Replacement

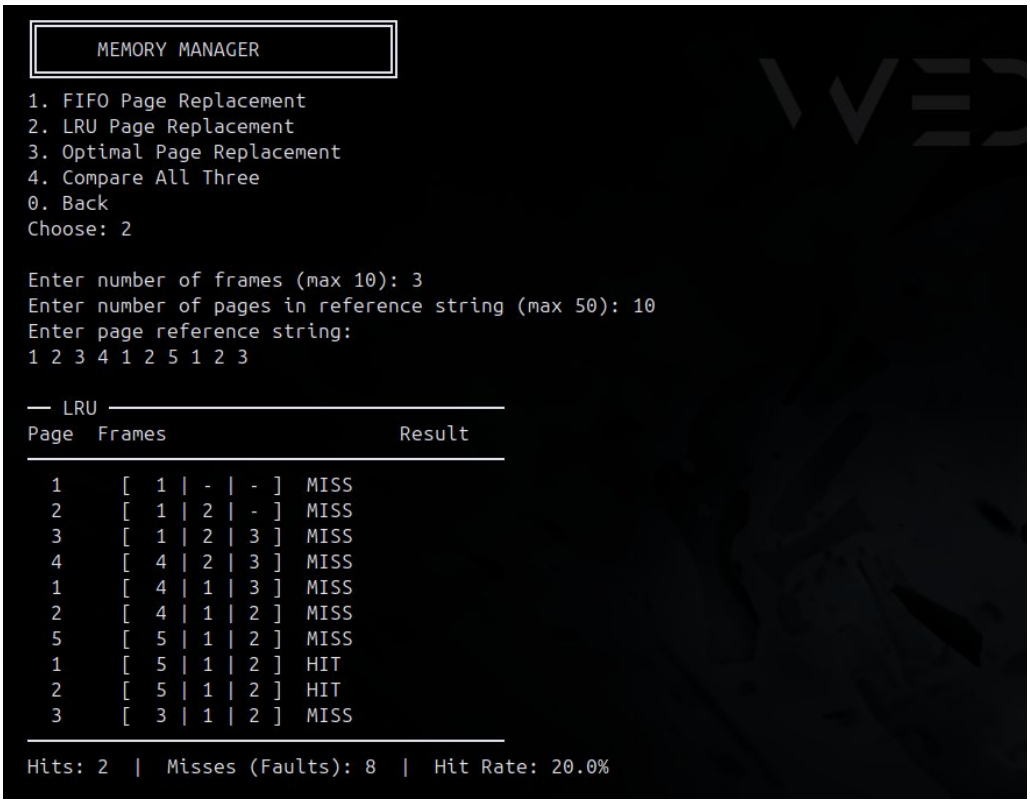


Figure 8.5: LRU Page Replacement Output

### 8.2.3 Optimal Page Replacement

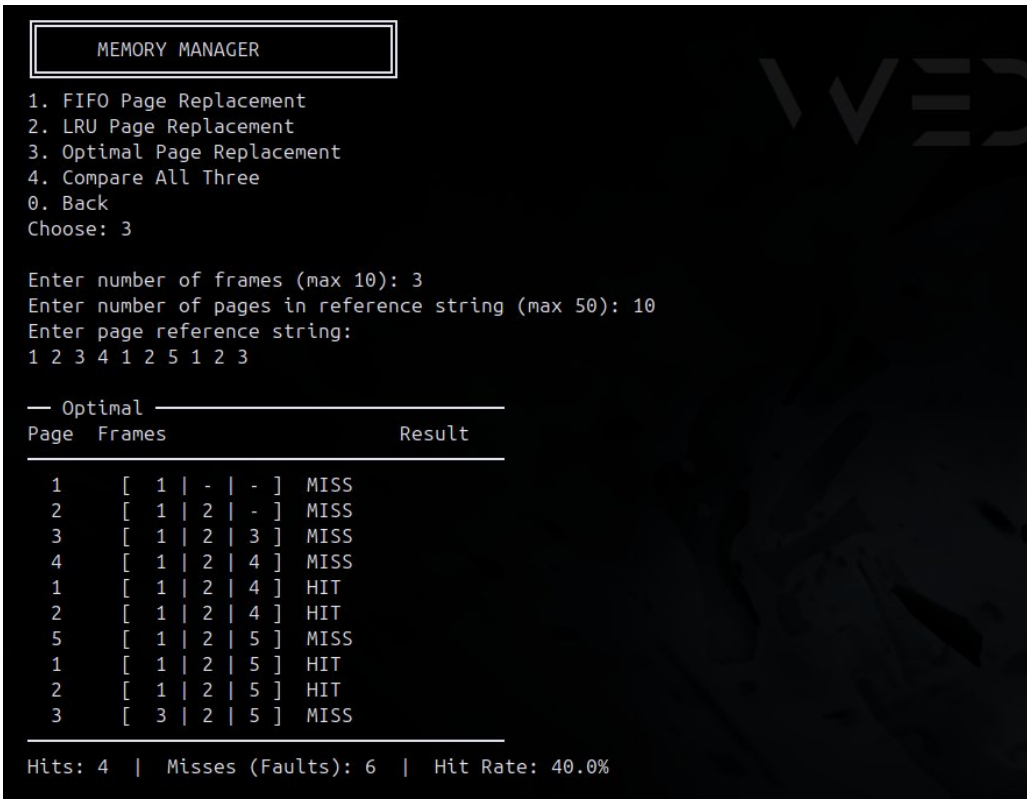


Figure 8.6: Optimal Page Replacement Output

### 8.2.4 Comparison of All Three Algorithms


Figure 8.7: Page Replacement Algorithm Comparison

As expected, Optimal produced the highest hit rate, followed by LRU, with FIFO performing worst. This matches the theoretical ordering: Optimal $\geq$ LRU $\geq$ FIFO for most reference strings.

## 8.3 Module 3 — IPC Producer-Consumer

The IPC module was tested with 2 producer threads, 2 consumer threads, and a buffer size of 5.

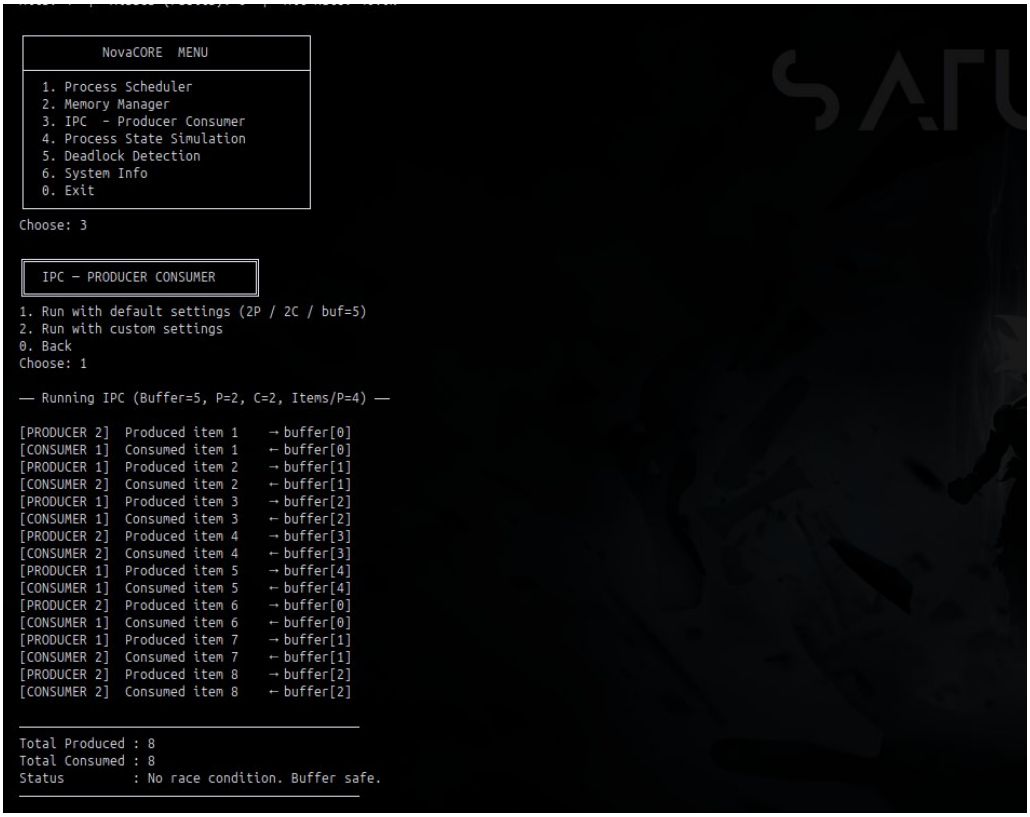


Figure 8.8: IPC Producer-Consumer Thread Output

The output confirms correct synchronization. No producer overwrote an unconsumed item, and no consumer read from an empty slot across all runs. The thread interleaving varies between runs (as expected with concurrent threads), but the total items produced always equalled the total items consumed.

## 8.4 Module 4 — Process State Simulation

The simulation was run with 4 processes with burst times: P1=3, P2=4, P3=5, P4=2.


Figure 8.9: Process State Simulation Cycle-by-Cycle Output

The simulation completed in 14 cycles. All five states (NEW, READY, RUNNING, WAITING, TERMINATED) were observed during the run. P1 and P3 both passed through the WAITING state due to their burst times hitting multiples of 3. P4 (burst=2) went directly from RUNNING to TERMINATED without ever entering WAITING, which is the correct behavior since 2 is not a multiple of 3 and it reaches 0 before hitting 3.

## 8.5 Module 5 — Deadlock Detection

The Banker's Algorithm was run on the classic preset example with 5 processes (P0–P4) and 3 resource types (A, B, C).

### 8.5.1 Resource State Table

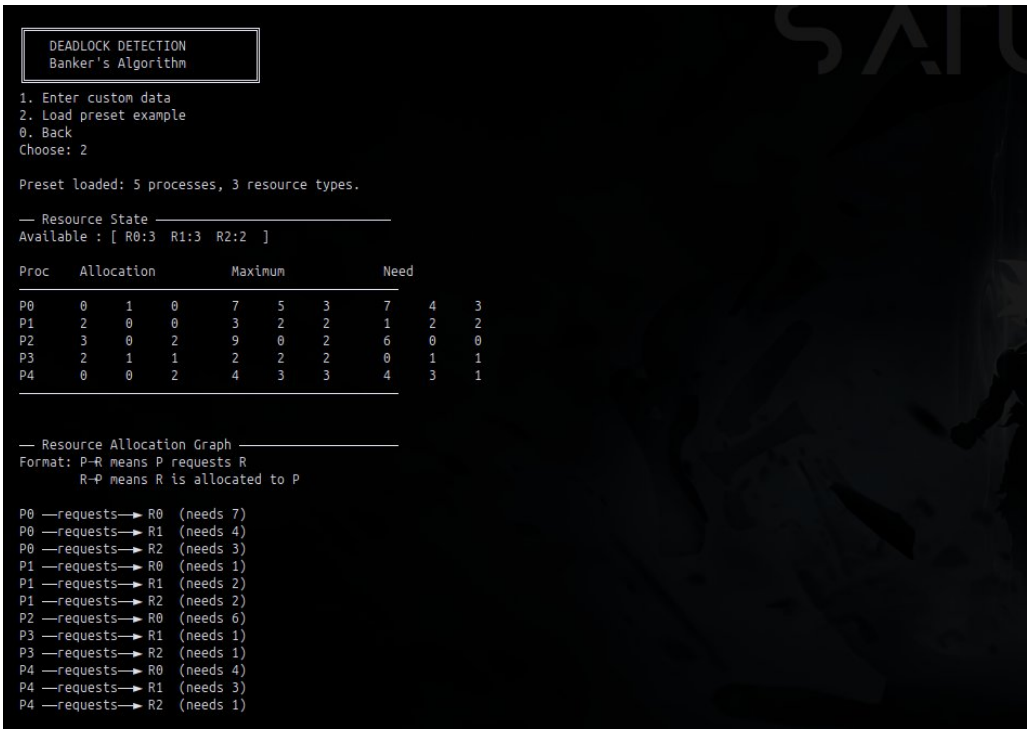


Figure 8.10: Deadlock Detection — Resource Allocation State

### 8.5.2 Safety Check and Safe Sequence

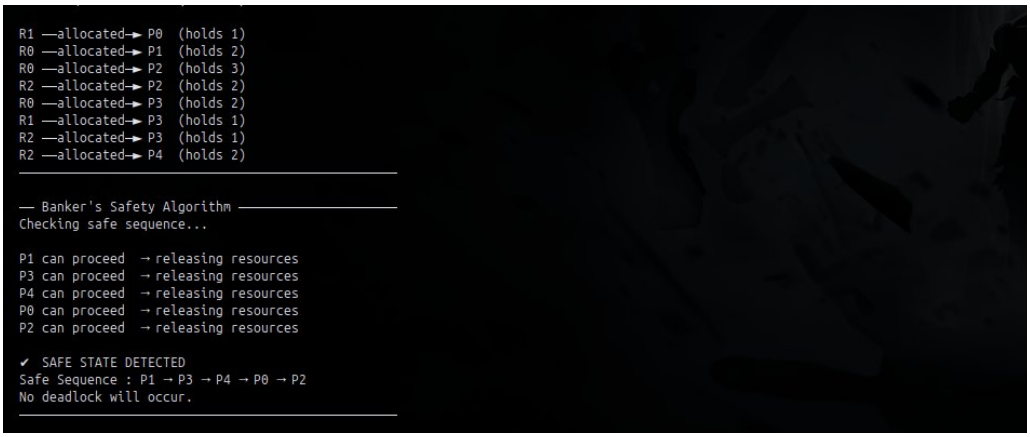


Figure 8.11: Deadlock Detection — Safety Algorithm Result

The algorithm correctly identified the system as being in a **SAFE STATE** and produced the safe sequence:

$$\mathbf{P1} \rightarrow \mathbf{P3} \rightarrow \mathbf{P4} \rightarrow \mathbf{P0} \rightarrow \mathbf{P2}$$

This matches the expected result from the textbook, confirming the implementation is correct.

### 8.5.3 Resource Allocation Graph

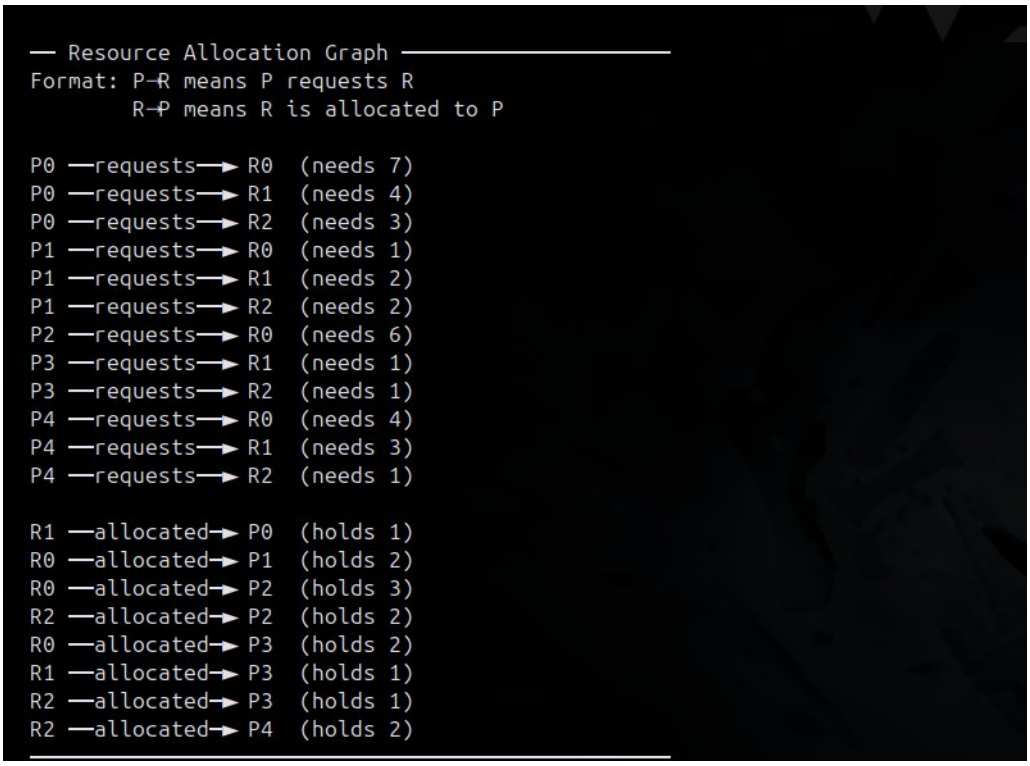


Figure 8.12: Text-Based Resource Allocation Graph

## 8.6 Module 6 — xv6 Kernel Extension

### 8.6.1 xv6 Boot

After running `setup_xv6.sh` and then `make qemu-nox` inside the `xv6-public` folder, xv6 boots successfully inside QEMU.

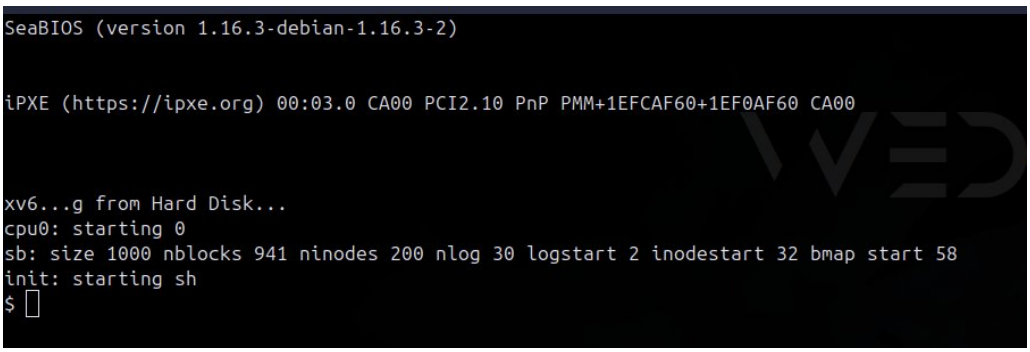


Figure 8.13: xv6 Boot Inside QEMU

### 8.6.2 meminfo Output

After booting, typing `meminfo` in the xv6 shell calls the custom `getmeminfo()` system call and prints live physical memory statistics from the running kernel.

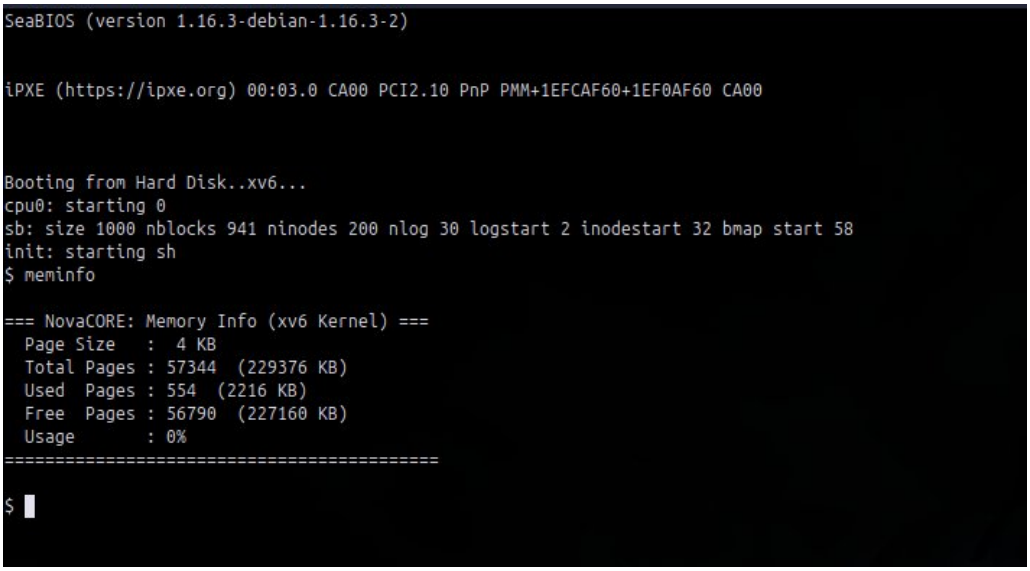


Figure 8.14: NovaCORE `meminfo` Output Inside xv6

The expected output is:

```
=== NovaCORE: Memory Info (xv6 Kernel) ===
  Page Size   :  4 KB
  Total Pages : 57344  (229376 KB)
  Used  Pages : 554    (2216 KB)
  Free  Pages : 56790  (227160 KB)
  Usage       : 0%
========================================
```

The 0% usage is expected behavior. At boot, only about 554 pages out of 57344 are used, which is less than 1%. Since integer division is used for the percentage calculation (which is standard for kernel-space arithmetic in xv6), this rounds down to 0%.

## 8.7 NovaCORE Main Menu

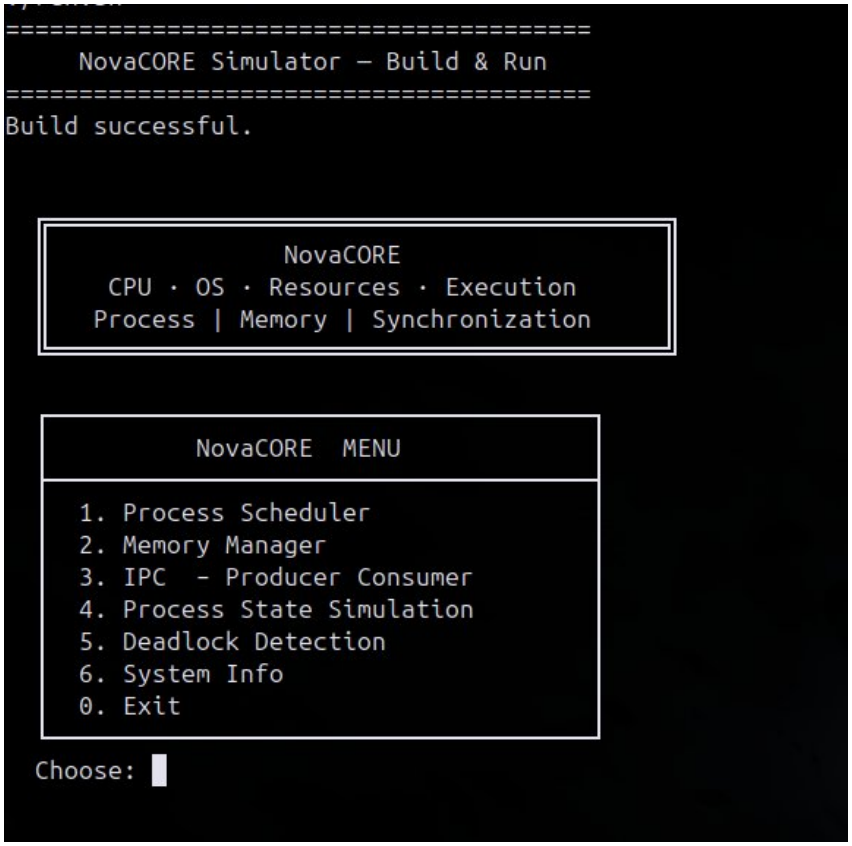


Figure 8.15: NovaCORE Main Menu

The main menu loads instantly on startup and provides access to all six modules through numbered options. The terminal box-drawing characters give it a clean, professional appearance in any Linux terminal.

# Chapter 9

# Challenges and Solutions

This chapter describes the real problems that came up during development and how each one was solved. These were not minor issues — some of them took significant time to debug and fix. Understanding these challenges is important because the solutions directly shaped how the final code works.

## 9.1 Challenge 1 — Round Robin Clock Initialization Bug

### 9.1.1 The Problem

The first version of the Round Robin scheduler initialized the CPU clock at time zero, regardless of when the first process actually arrived. When all processes had arrival times greater than zero (for example, arriving at times 2, 3, and 4), the scheduler would calculate negative waiting times for the first few processes.

For example, if a process arrived at time 2 and completed at time 5, the Turnaround Time was correctly 3. But Waiting Time was calculated as TAT minus Burst Time. If the process started running immediately at time 0 instead of time 2, the completion time was wrong, and the final waiting time came out negative. This is mathematically impossible and means the simulator was giving incorrect results.

### 9.1.2 The Solution

The fix was to initialize the clock to the arrival time of the first process (after sorting by arrival time), not to zero:

```
// Before fix (wrong):
int time = 0;

// After fix (correct):
int time = procs[0].arrival_time;
for (int i = 0; i < n; i++)
    if (procs[i].arrival_time <= time) {
        queue[rear++] = i;
        in_queue[i] = 1;
```

```
    }
```

Listing 9.1: Bug fix: clock starts at first arrival

This change also required adding an initial enqueue step that adds all processes which have already arrived by the starting time before the main loop begins. Without this, the queue starts empty and the first process never gets picked up correctly.

## 9.2 Challenge 2 — xv6 Not Compiling on GCC 13

### 9.2.1 The Problem

The stock xv6 source code from MIT was written to compile with older versions of GCC. When we tried to build it on Ubuntu 24.04 with GCC 13, the build failed with multiple warnings that were treated as errors. The two specific issues were:

- `-Werror=array-bounds`: GCC 13 added stricter bounds checking on array accesses. Some patterns in the xv6 source triggered false positives on this check even though the code was functionally correct.
- `-Werror=infinite-recursion`: GCC 13 also added detection for functions that appear to call themselves unconditionally. One xv6 pattern triggered this check even though it was not actually an infinite recursion at runtime.

Both of these caused the build to stop completely, meaning we could not even get to the stage of testing our kernel extension.

### 9.2.2 The Solution

The solution was to add two warning suppression flags to the xv6 Makefile:

```
CFLAGS += -Wno-array-bounds
CFLAGS += -Wno-infinite-recursion
```

Listing 9.2: GCC 13 compatibility flags added to xv6 Makefile

These flags do not disable error checking globally. They only suppress these two specific false-positive warnings in the xv6 source. All of our own added code (the `getmeminfo()` syscall, `kalloc_free_pages()`, and `meminfo.c`) compiled cleanly without needing these flags.

This fix was added automatically by the `setup_xv6.sh` script using `sed` so that anyone running the setup script gets a working build without having to manually edit the Makefile.

## 9.3 Challenge 3 — Kernel Pointer Validation with argptr()

### 9.3.1 The Problem

When implementing `sys_getmeminfo()`, the first version directly cast the user-space pointer to a kernel pointer:

```
// WRONG - unsafe, ignores address space boundary
int sys_getmeminfo(void) {
    struct meminfo *mi = (struct meminfo *)arg0();
    mi->free_pages = kalloc_free_pages();
    return 0;
}
```

Listing 9.3: Incorrect kernel pointer handling (first version)

This approach is dangerous. In xv6, user-space and kernel-space memory are separate. A user program could pass any address — including an invalid one, a kernel address, or a null pointer — and this code would write directly to it without any validation. This could crash the kernel or corrupt kernel memory.

### 9.3.2 The Solution

The correct xv6 approach is to use `argptr()`, which is the standard kernel function for safely translating and validating user-space pointers before the kernel uses them:

```
int sys_getmeminfo(void) {
    struct meminfo *mi;
    if (argptr(0, (void*)&mi, sizeof(*mi)) < 0)
        return -1;
    mi->free_pages  = kalloc_free_pages();
    mi->total_pages = PHYSTOP / PGSIZE;
    mi->used_pages  = mi->total_pages - mi->free_pages;
    return 0;
}
```

Listing 9.4: Correct kernel pointer handling using argptr()

`argptr()` checks that the pointer points to a valid user-space address, that the full size of the struct fits within user-space memory, and that the address is not null. If any of these checks fail, it returns a negative value and the syscall returns -1 to the user program safely. This is the same pattern used by all other system calls in xv6 that accept pointers from user programs.

## 9.4 Challenge 4 — IPC Race Condition in Early Version

### 9.4.1 The Problem

The first version of the IPC module used a single mutex to protect the entire buffer. While this prevented data corruption, it also meant that producers and consumers could

never run at the same time. A producer would lock the mutex, write an item, then unlock. Only then could a consumer lock it and read. This defeated the purpose of using multiple threads.

More importantly, the first version also had a subtle race condition in the buffer index management. The `in_idx` (write pointer) and `out_idx` (read pointer) were being incremented outside the mutex, which meant two producers could read the same index value before either of them incremented it, causing them to write to the same slot.

### 9.4.2 The Solution

Two changes were made:

1. **Separate mutexes for producers and consumers.** Producers lock a producer mutex when accessing `in_idx`, and consumers lock a consumer mutex when accessing `out_idx`. Since these are separate pointers, producers and consumers can now operate in parallel without blocking each other.
2. **Index updates inside the mutex.** Both `in_idx` and `out_idx` are now read and incremented while the respective mutex is held, preventing two threads from ever getting the same index value.

The semaphores (`empty_sem` and `full_sem`) handle the higher-level coordination of how many items are in the buffer at any time. The mutexes only protect the index pointers. This two-layer design is the standard correct pattern for the bounded buffer problem.

## 9.5 Challenge 5 — Optimal Page Replacement Look-Ahead Logic

### 9.5.1 The Problem

The Optimal page replacement algorithm requires looking ahead in the reference string to decide which page will not be used for the longest time. The tricky part is handling the case where a page currently in memory never appears again in the remaining reference string.

In the first version, the look-ahead loop returned `page_count` (the length of the entire reference string) as the "distance" for pages that never appear again. This worked most of the time, but failed when all frames contained pages that appeared later in the string. In that case, the algorithm still had to pick one of them, but the comparisons were not handled correctly because all of them had distance equal to `page_count`, causing the selection logic to behave inconsistently.

### 9.5.2 The Solution

The fix was to use `INT_MAX` (the maximum integer value) as the sentinel for "never appears again" instead of `page_count`:

```
for (int j = 0; j < frames; j++) {
    int next_use = INT_MAX; // assume never used again
    for (int k = i + 1; k < page_count; k++)
```

```
        if (pages[k] == frame[j]) {
            next_use = k;
            break;
        }
    if (next_use > farthest) {
        farthest = next_use;
        idx = j;
    }
}
```

Listing 9.5: Optimal look-ahead with INT_MAX sentinel

Using `INT_MAX` ensures that any page that never appears again will always be selected for replacement over any page that appears at a finite future position. This gives the true optimal replacement decision in all cases.

## 9.6 Challenge 6 — Process State Simulation Infinite Loop

### 9.6.1 The Problem

During testing of the process state simulation with certain burst time combinations, the simulation would not terminate. This happened when a process with a burst time that was a multiple of 3 (for example, burst=6) kept triggering the I/O wait condition every time it resumed running, because after each 1-cycle execution its remaining burst would always hit a multiple of 3 again.

For burst=6: runs 1 cycle (remaining=5, not multiple of 3), runs again (remaining=4, not multiple), runs again (remaining=3, multiple of 3 → WAITING), resumes after 2 cycles, runs 1 cycle (remaining=2, not multiple), runs again (remaining=1, not multiple), runs again (remaining=0 → TERMINATED). This case actually terminates fine. But for burst=9: remaining hits 6 (WAITING), then 3 (WAITING again), which works, but in more complex combinations with multiple processes the loop counter was still needed as a safety measure.

### 9.6.2 The Solution

A 60-cycle safety cap was added to the simulation loop. If the simulation has not finished after 60 cycles, it stops and prints a notice. In practice, all normal inputs finish well within 60 cycles. The cap only activates if there is an unexpected combination of burst times and I/O triggers that would otherwise cause the simulation to run indefinitely:

```
if (++cycle > 60) {
    printf("\n  [Simulation capped at 60 cycles]\n");
    break;
}
```

Listing 9.6: 60-cycle safety cap

# Chapter 10

# Individual Contribution

This chapter describes what each group member worked on and contributed to the NovaCORE project. The work was divided by module so that each person had clear ownership and responsibility over their part.

## 10.1 Muhammad Ibrahim — 01-135241-035

**Role: Process Scheduler + Main Menu + Project Integration**

Muhammad Ibrahim was responsible for the process scheduler module and the overall project entry point. His contributions included:

- Designed and implemented the full `main.c` entry point, including the main menu loop, the NovaCORE banner, and the system info display.
- Implemented all three scheduling algorithms in `scheduler.c`: Round Robin, SJF Non-Preemptive, and Priority Non-Preemptive.
- Identified and fixed the Round Robin clock initialization bug that was causing negative waiting times when processes arrived after time zero.
- Wrote the process input routine and the formatted results table that is shared across all three scheduler algorithms.
- Set up the project `Makefile` and `run.sh` build script.
- Handled overall project integration — merging all modules into a single compiling binary and ensuring the menu correctly called each module's entry function.
- Verified all scheduler outputs manually against paper calculations before final submission.

## 10.2 Manzer Bibi — 01-135241-026

**Role: Memory Manager**

Manzer Bibi was responsible for the memory manager module. Her contributions included:

- Implemented all three page replacement algorithms in `memory.c`: FIFO, LRU, and Optimal.
- Designed the frame state display that shows the current contents of all frames after each page reference, along with the HIT or MISS result.
- Implemented the Compare All Three mode that runs FIFO, LRU, and Optimal on the same input and prints a side-by-side summary of hits, misses, and hit rates.
- Fixed the Optimal algorithm look-ahead bug by replacing the `page_count` sentinel with `INT_MAX`, which correctly handles pages that never appear again in the reference string.
- Verified all page replacement outputs against manually traced reference string examples.
- Wrote the `memory.h` header file and the `memory_menu()` function that connects the module to the main menu.

## 10.3 Abdul Wahab — 01-135241-001

**Role: Deadlock Detection + Process State Simulation**

Abdul Wahab was responsible for two modules: the Banker's Algorithm deadlock detector and the process state simulation. His contributions included:

- Implemented the full Banker's Algorithm in `deadlock.c`, including the Need matrix computation, input validation, the safety algorithm loop, and the safe sequence output.
- Implemented the text-based Resource Allocation Graph that shows request edges (P → R) and assignment edges (R → P) for all processes and resources.
- Added the preset example mode using the classic 5-process, 3-resource textbook scenario, which produces the safe sequence P1 → P3 → P4 → P0 → P2.
- Implemented the five-state process simulation in `process_state.c`, including the I/O trigger logic (burst remaining hits a multiple of 3), the 2-cycle WAITING countdown, and the simple dispatcher that picks the next READY process.
- Added the 60-cycle safety cap to prevent the simulation from running indefinitely on edge-case inputs.
- Wrote `deadlock.h`, `process_state.h`, and both module menu functions.

## 10.4 Ateeq Ur Rehman — 01-135241-011

**Role: IPC Producer-Consumer**

Ateeq Ur Rehman was responsible for the IPC module. His contributions included:

- Implemented the Producer-Consumer bounded buffer problem in `ipc.c` using POSIX threads (`pthreads`), POSIX semaphores, and mutexes.

- Designed the two-semaphore synchronization model using `empty_sem` and `full_sem` to coordinate producers and consumers without busy-waiting.
- Fixed the race condition in the early version by separating the producer mutex and consumer mutex, and moving the buffer index updates inside the mutex lock.
- Implemented both the default mode (2 producers, 2 consumers, buffer size 5) and the custom mode where the user can configure the number of producers, consumers, buffer size, and items per producer.
- Wrote the thread activity log output that prints each produce and consume event in real time with thread ID, item value, and buffer index.
- Wrote `ipc.h` and the `ipc_menu()` function.

## 10.5 Saim Zafar — 01-135241-045

**Role: xv6 Kernel Extension**

Saim Zafar was responsible for the xv6 kernel extension. His contributions included:

- Researched the xv6 system call registration process by studying the existing syscalls in the xv6 source (particularly `sys_write`, `sys_read`, and `sys_sbrk`) to understand the correct pattern to follow.
- Added the `struct meminfo` definition to `proc.h` so it is accessible to both the kernel and user-space code.
- Implemented `kalloc_free_pages()` in `kalloc.c`, which walks the kernel free page linked list under the memory lock to count available physical pages.
- Implemented `sys_getmeminfo()` in `sysproc.c` using `argptr()` for safe user-space pointer validation, replacing an earlier unsafe direct-cast version.
- Registered syscall number 22 across all required files: `syscall.h`, `syscall.c`, `user.h`, `usys.S`, and `defs.h`.
- Wrote the `meminfo.c` user utility that calls `getmeminfo()` and prints the formatted memory statistics output.
- Diagnosed and fixed the GCC 13 compilation issue by adding `-Wno-array-bounds` and `-Wno-infinite-recursion` to the xv6 Makefile.
- Wrote the complete `setup_xv6.sh` automation script that clones xv6, applies all patches, copies `meminfo.c`, and builds the kernel in one step.

# Chapter 11

# Conclusion

NovaCORE was built to solve a real problem that most OS students face: the gap between learning theory in class and actually seeing algorithms run on real inputs. Over the course of this project, that gap was closed in a practical and hands-on way.

## 11.1 What Was Accomplished

The project delivered two working systems:

**The Linux Simulator** is a fully interactive terminal application written in C that covers six core OS concepts in one place. A user can enter their own process data and run Round Robin, SJF, or Priority scheduling and immediately see Completion Time, Turnaround Time, and Waiting Time calculated correctly. They can test page replacement algorithms on any reference string and compare FIFO, LRU, and Optimal side by side. They can watch Producer-Consumer synchronization work in real time with live thread output. They can see process state transitions happen cycle by cycle. And they can run the Banker's Algorithm on any resource allocation scenario and find the safe sequence if one exists.

**The xv6 Kernel Extension** goes further than simulation. A real system call — `getmeminfo()`, registered as syscall number 22 — was added to the xv6 teaching kernel by modifying nine kernel source files. A user-space utility called `meminfo` was written and compiled into the xv6 filesystem. When run inside QEMU, it calls the syscall, the kernel walks its own free page linked list, and live physical memory statistics are returned to user space and printed. This is not a simulation of kernel behavior — it is actual kernel-level programming producing real output from inside a running OS.

The entire project compiles cleanly with zero warnings on GCC 13 with `-Wall -std=c99`.

## 11.2 What Was Learned

Building NovaCORE taught things that cannot be learned from a textbook alone.

On the **algorithm side**, implementing Round Robin revealed edge cases that paper examples never show — like what happens when the CPU is idle and needs to jump forward

to the next arrival, or how the clock starting point affects every single timing calculation. Implementing Optimal page replacement showed why looking ahead is computationally expensive and why real systems cannot use it. Implementing the Banker's Algorithm made the safety check loop feel natural rather than abstract.

On the **systems programming side**, the IPC module showed that getting semaphore and mutex logic correct is harder than it looks. A single misplaced lock or unlock can cause a deadlock or a race condition that only appears occasionally and is difficult to reproduce. The xv6 work showed how system calls actually work at the kernel level — how a function call in user space turns into a trap, gets dispatched through a table, executes kernel code, and returns a result safely across the user-kernel boundary.

On the **debugging side**, every challenge described in Chapter 10 was a genuine learning experience. The Round Robin clock bug, the GCC 13 compatibility issue, the `argptr()` pointer validation requirement, the IPC race condition — each one required understanding the problem deeply before the fix made sense.

## 11.3 Course Learning Outcomes

NovaCORE directly satisfies all three course learning outcomes for CSC320:

- **CLO1 — Apply fundamental OS concepts and system-level programming:** The simulator applies scheduling, memory management, synchronization, and deadlock concepts in working C code. The xv6 extension applies system-level programming at the kernel level.
- **CLO2 — Implement process management, memory management, and synchronization:** All three areas are implemented from scratch. Process management covers scheduling and state simulation. Memory management covers three page replacement algorithms. Synchronization covers the Producer-Consumer problem with semaphores and mutexes.
- **CLO3 — Design and develop OS-based solutions using appropriate algorithms and techniques:** NovaCORE is a complete, designed solution — not a collection of disconnected scripts. The two-layer architecture (simulator + kernel extension) and the unified menu system reflect deliberate design decisions. The algorithms chosen (RR, SJF, Priority, FIFO, LRU, Optimal, Banker's) are the standard OS algorithms taught in the course and appropriate for the problem.

## 11.4 Limitations and Future Work

NovaCORE covers the major OS topics but there are areas that could be extended in future work:

- **Preemptive scheduling:** The current scheduler only implements non-preemptive SJF and Priority. Preemptive versions (SRTF and Preemptive Priority) would make the simulator more complete.
- **More IPC mechanisms:** The current IPC module only covers the Producer-Consumer problem. Adding message passing or shared memory segments would

demonstrate a wider range of IPC techniques.

- **Graphical output:** The simulator currently produces text-only output. A simple terminal-based bar chart for Gantt schedule visualization would make the scheduling results easier to read.
- **xv6 extension:** The current kernel extension only adds one system call. Future work could add a process listing syscall (similar to `ps`) or a simple virtual memory statistics call.

## 11.5 Final Remarks

NovaCORE started as a course requirement and turned into something that genuinely works and is useful. Every module produces correct, verifiable output. The xv6 extension runs inside a real kernel. The code is clean, well-commented, and compiles without warnings.

More importantly, the process of building it — debugging the clock bug, fixing the race condition, figuring out `argptr()`, getting xv6 to compile on GCC 13 — taught more about operating systems than any amount of reading could have. That was the point of the project, and in that sense it was a success.